



AI VIET NAM

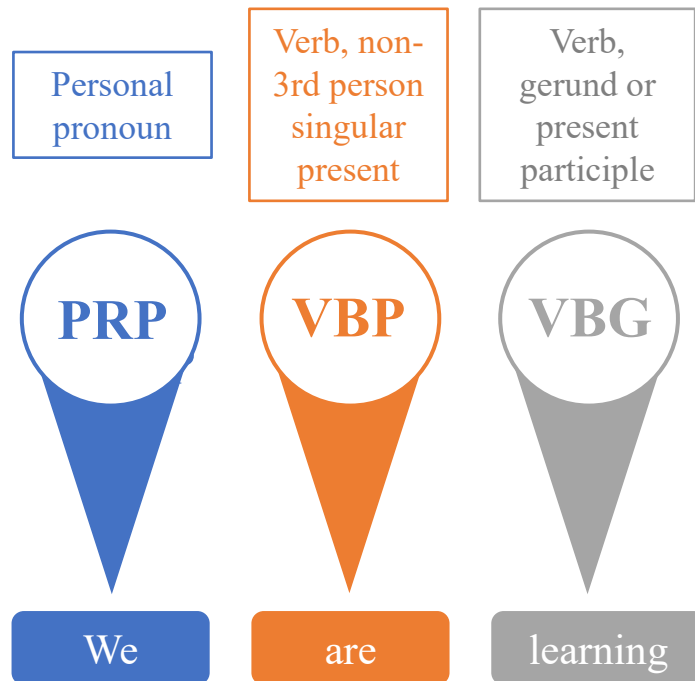
@aivietnam.edu.vn

From POS Tagging to NER

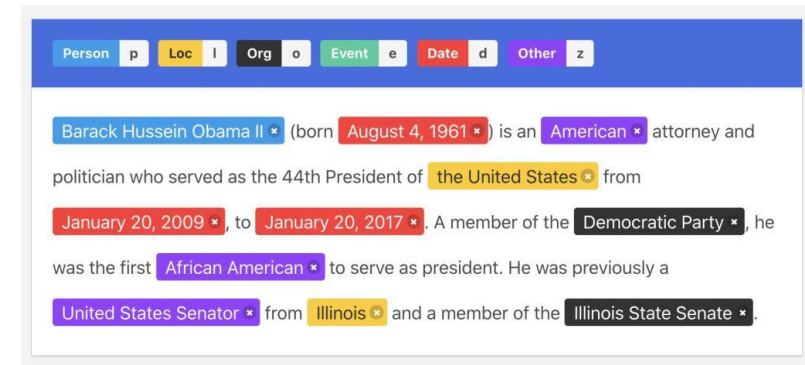
Dang Nha Nguyen – TA

Objectives

❖ POS Tagging



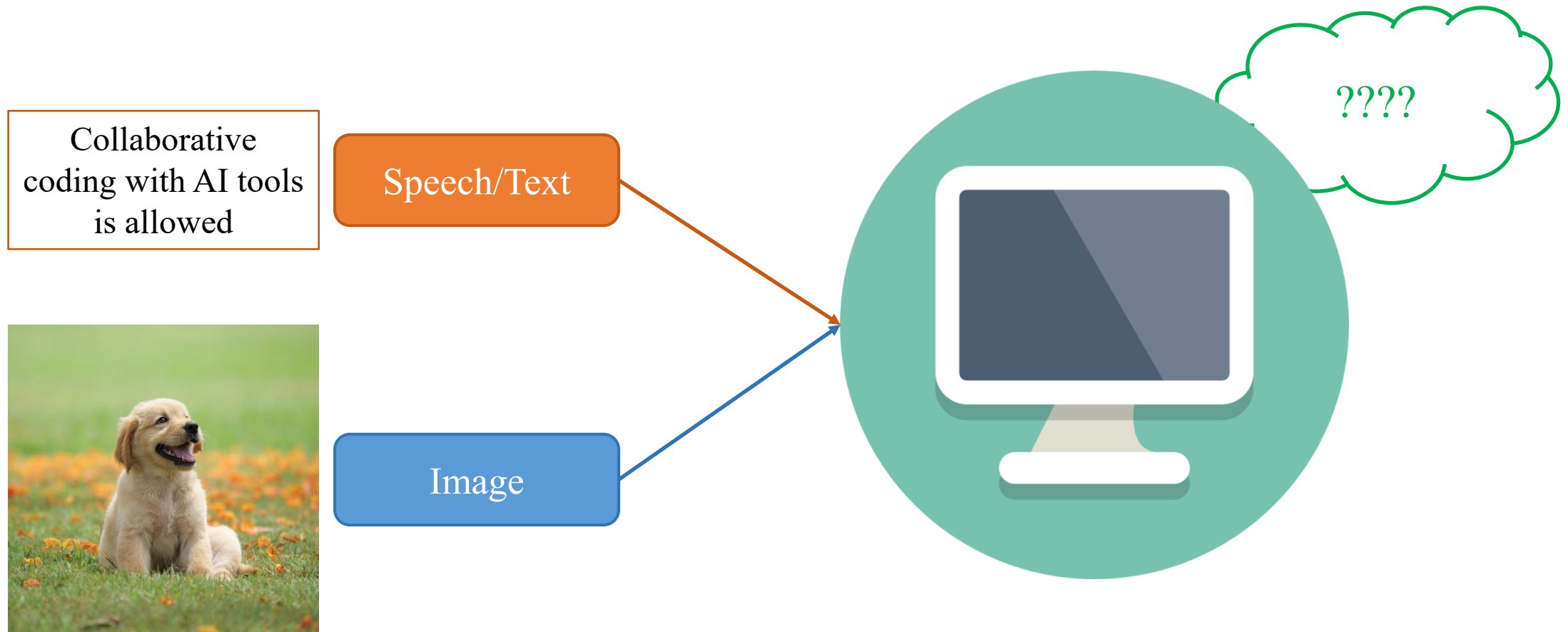
❖ NER



Overview

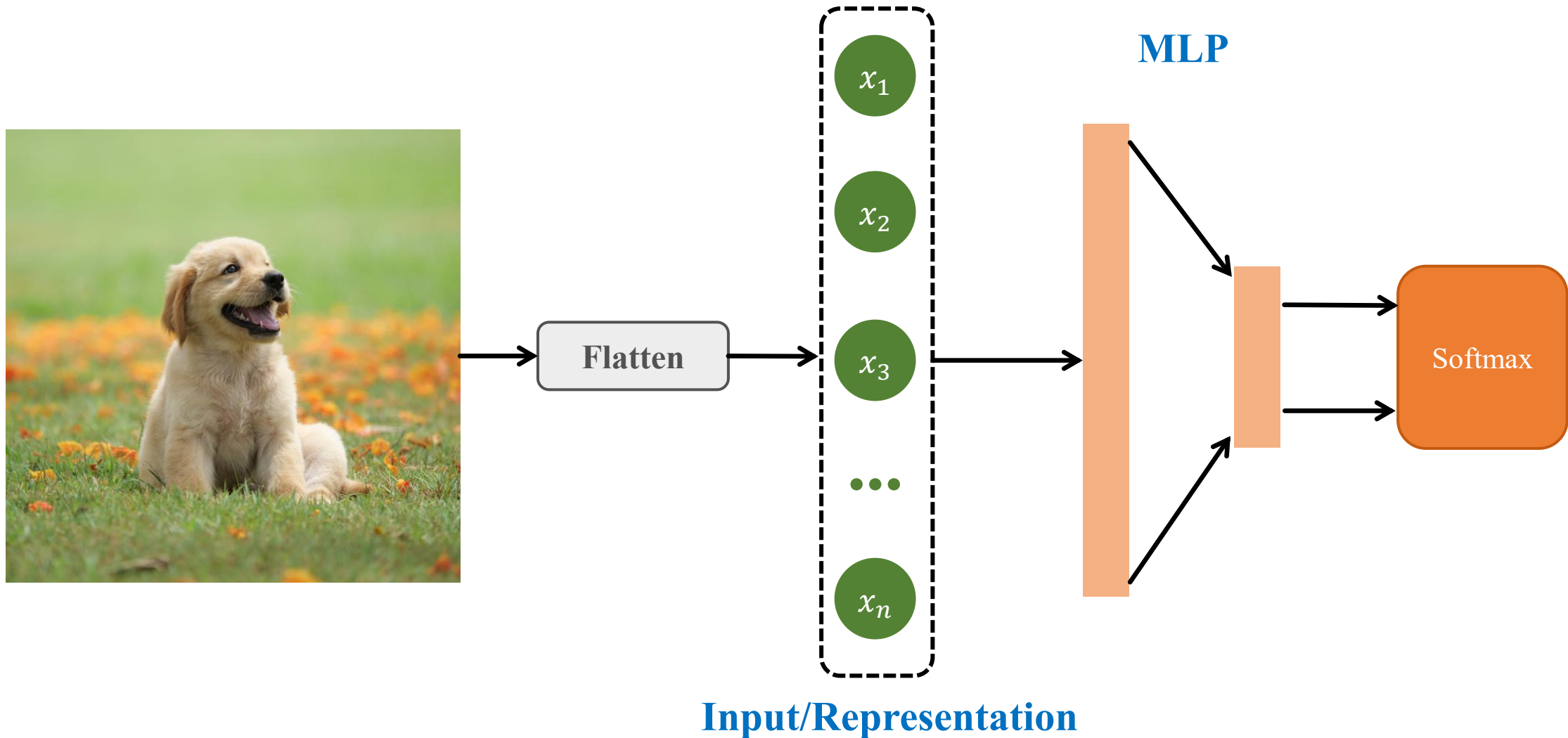
Introduction to NLP

❖ Human vs Computer



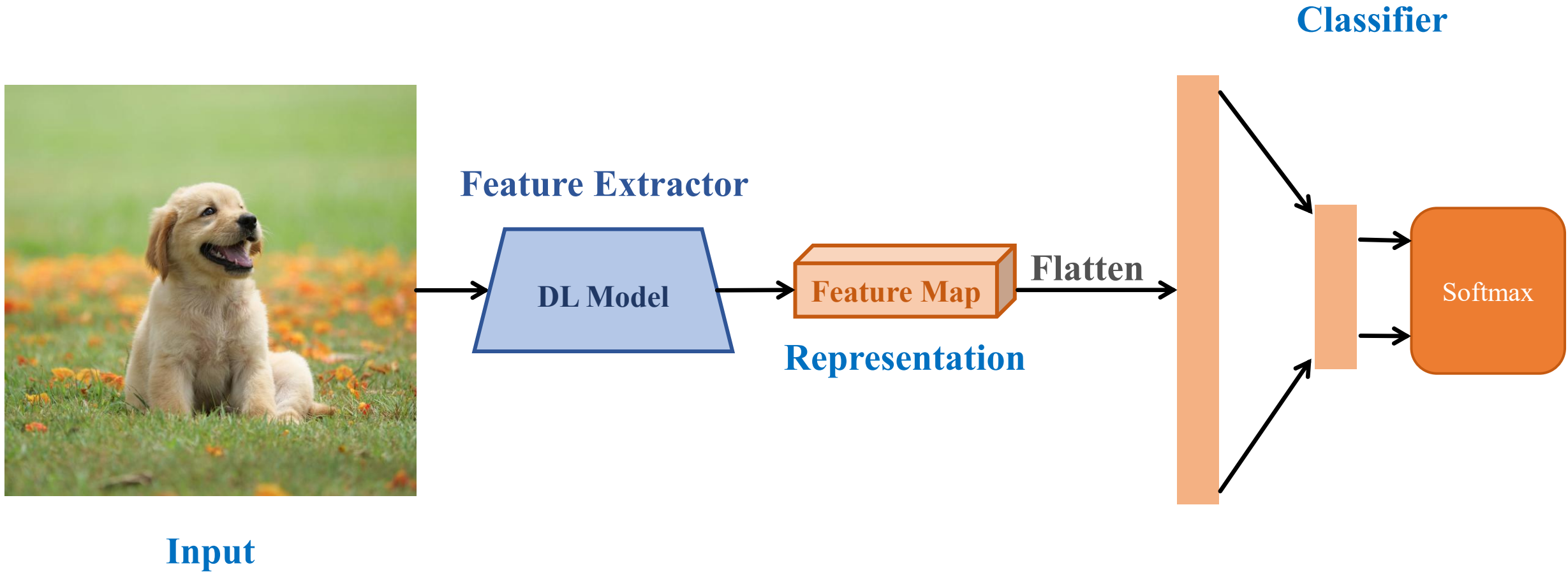
Overview

❖ Vision in Computer



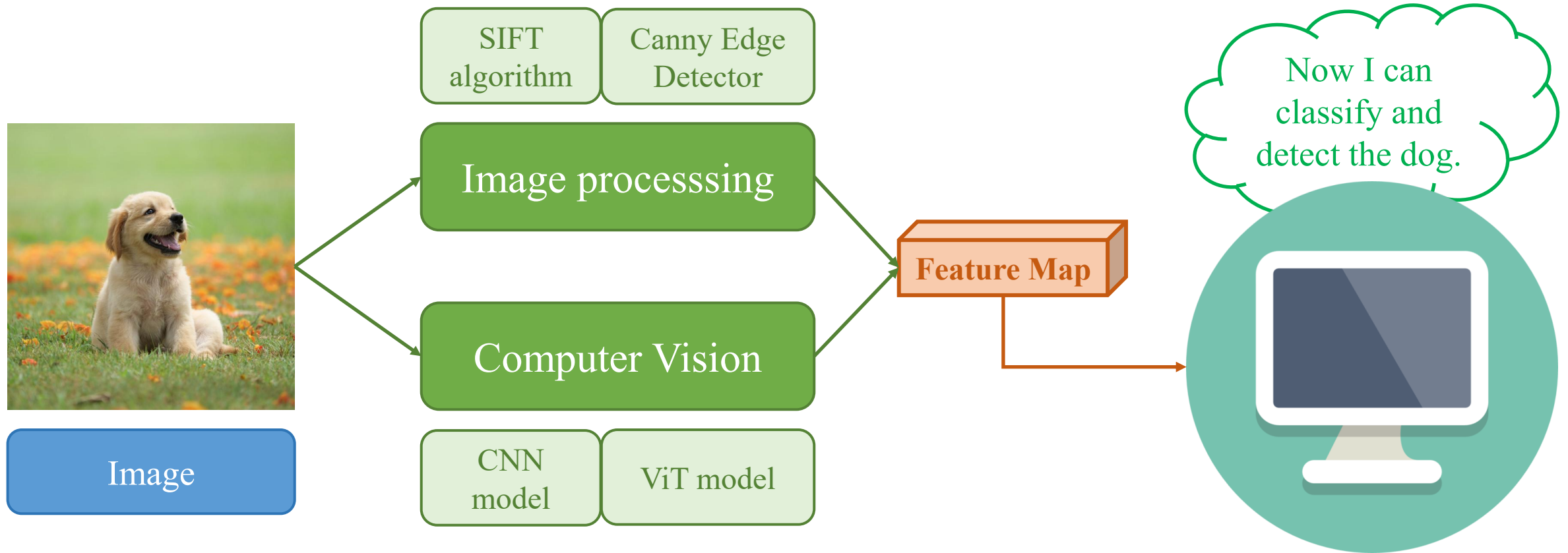
Overview

❖ Vision in Computer



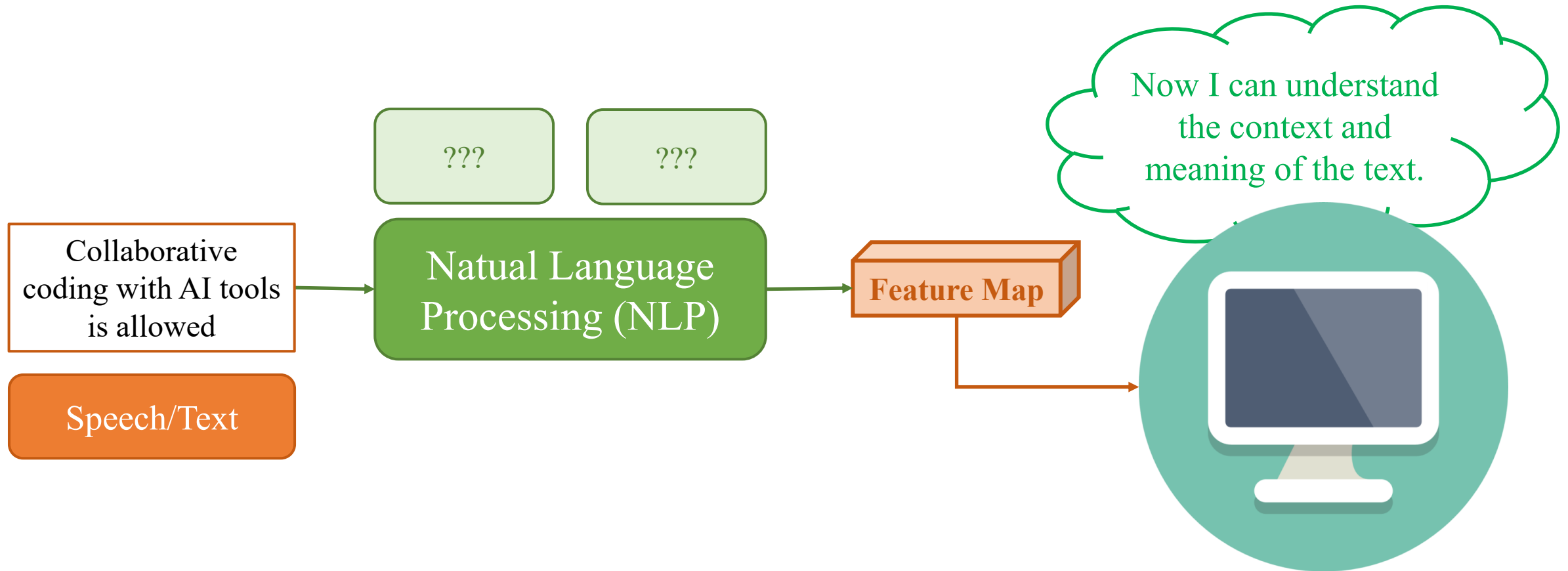
Overview

❖ Vision in Computer



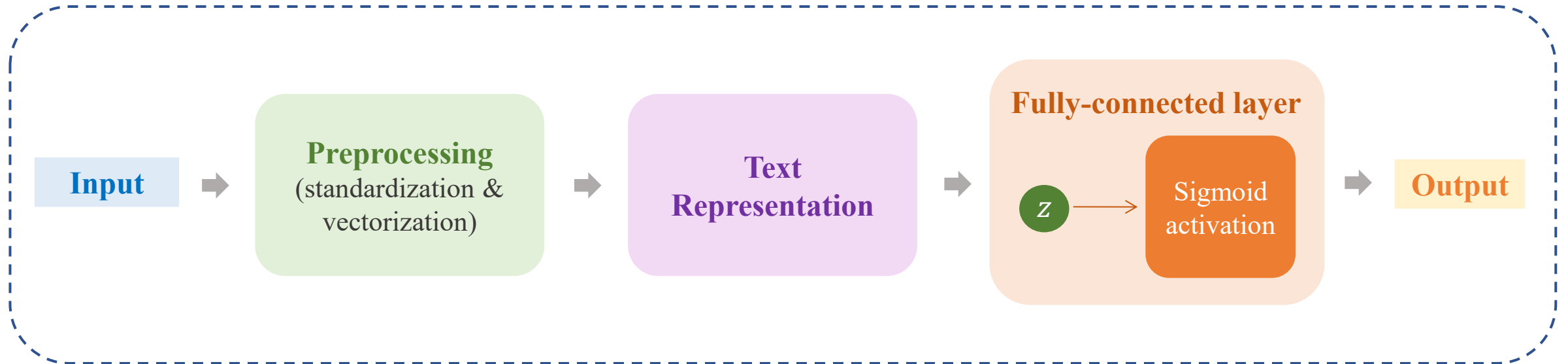
Overview

❖ Natural Language in Computer



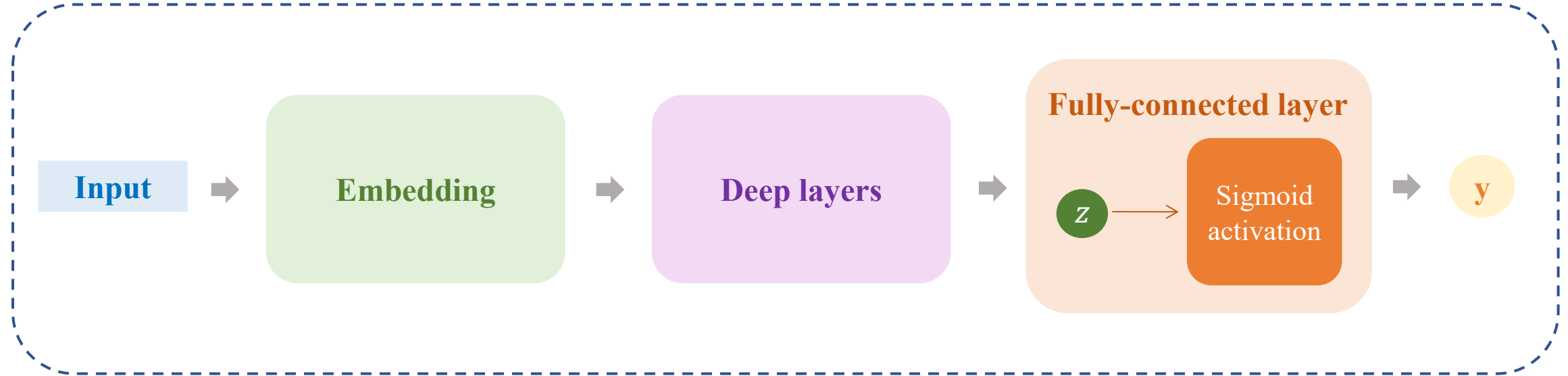
Overview

❖ Basic NLP pipeline



Overview

❖ Sentence Classification



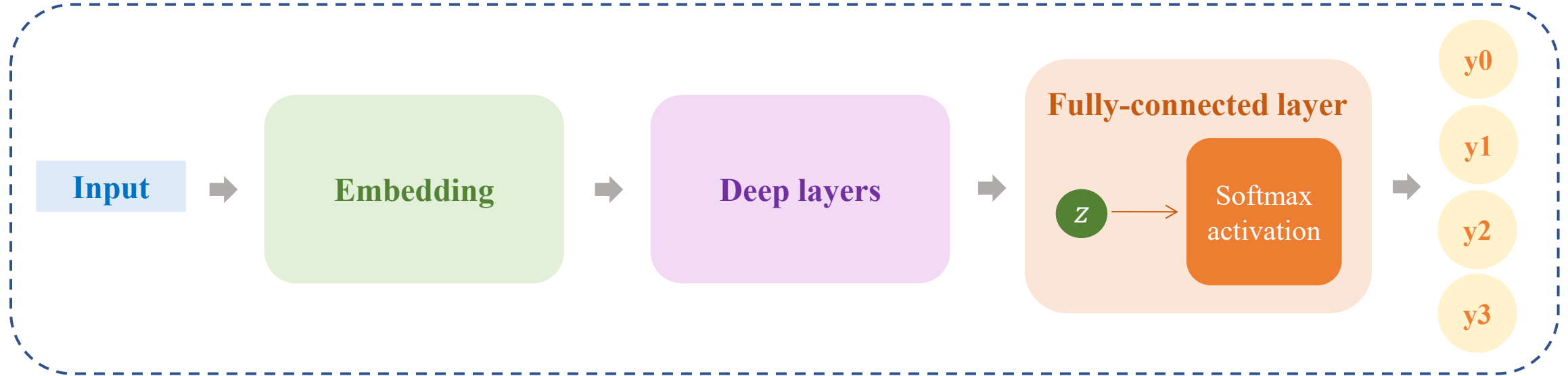
Text Classification

Sentiment Analysis

Spam/ Not Spam

Overview

❖ Token Classification



POS Tagging

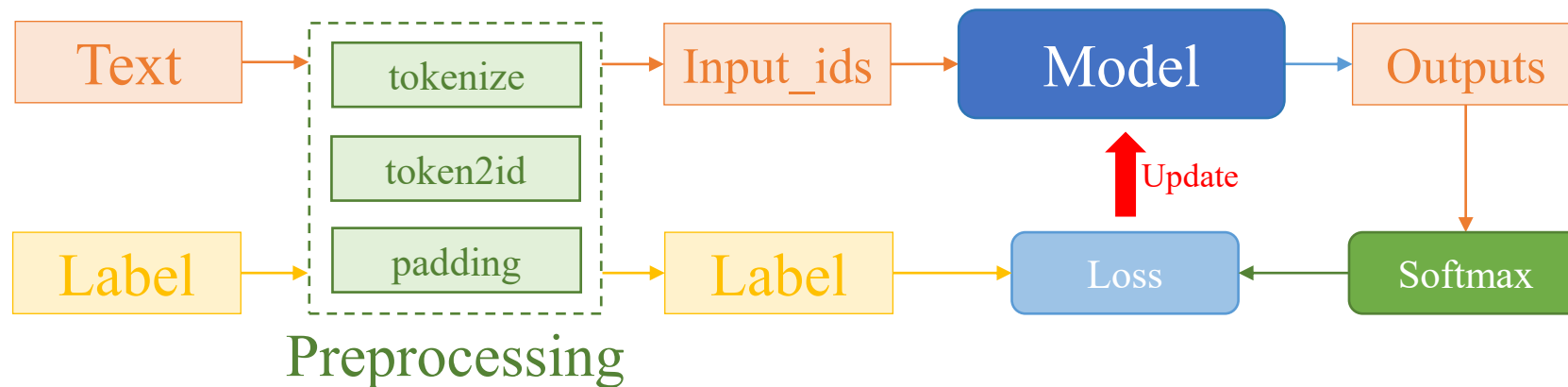
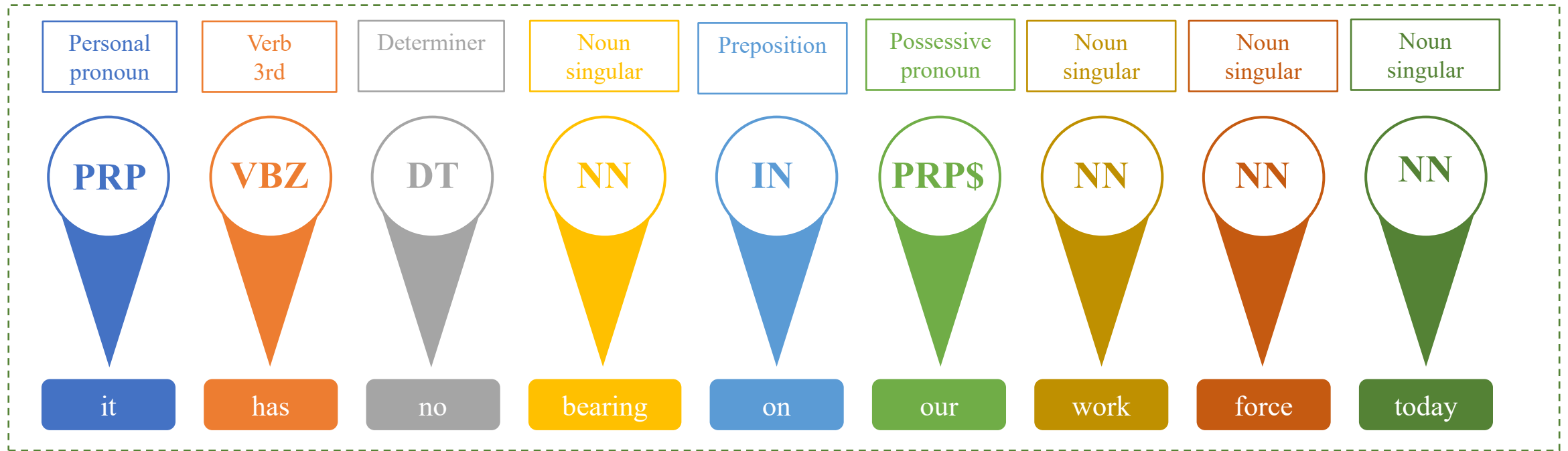
Named-entity recognition

Part-of-Speech Tagging

Part-of-Speech Tagging

- Preprocessing
- Embedding
- POS Tagging

Part-of-Speech Tagging



Preprocessing

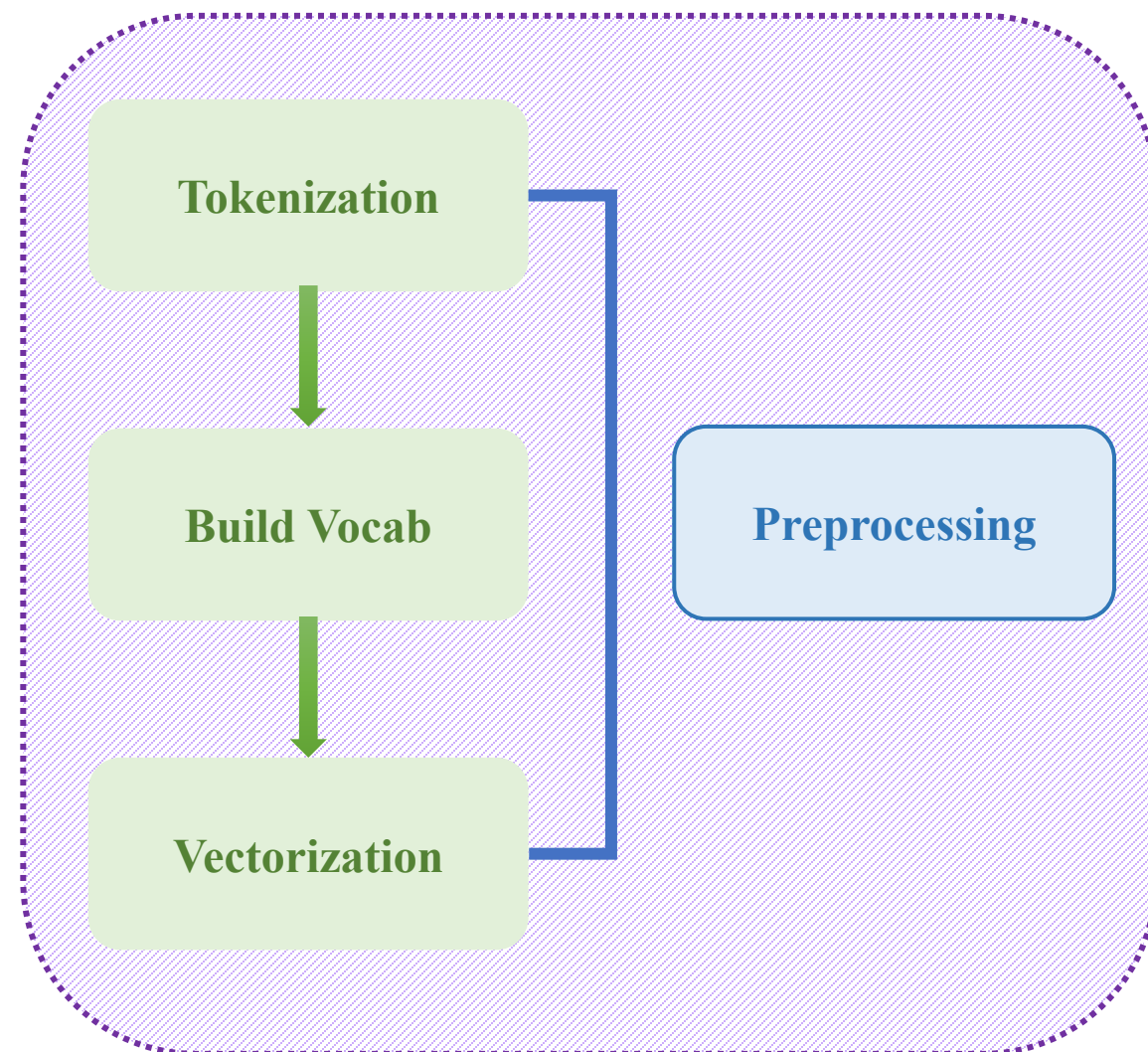
❖ Text Corpus

I like a dog

Books are expensive

```
corpus = [  
    "i like a dog",  
    "books are expensive"  
]
```

```
# 0: noun/pronoun - 1: verb - others - 2  
labels = [[0, 1, 2, 0],  
          [0, 1, 2]]
```



Tokenization

Example corpus

sample1: 'I like a dog'

sample2: 'Books are expensive'

I like a dog

Books are expensive



Tokenize

i like a dog

books are expensive

0 1 2 0

0 1 2

Label



```
sample1 = corpus[0]  
sample2 = corpus[1]
```

```
sample1_token = tokenizer.encode(sample1)  
print(sample1_token.tokens)
```

```
sample2_token = tokenizer.encode(sample2)  
print(sample2_token.tokens)
```

```
... ['i', 'like', 'a', 'dog']  
    ['books', 'are', 'expensive', '<pad>']
```

Building Vocab

```
vocab = tokenizer.get_vocab()  
print(vocab)
```

```
{'like': 8, 'expensive': 6, 'i': 7, 'are': 3, '<pad>': 1, 'a': 2, 'books': 4, 'dog': 5, '<unk>': 0}
```



0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary

Initialize
Tokenizer

Initialize
Tokenization

Handle
Padding&Truncation

Define vocab_size,
special_tokens

Build Vocab From
Corpus

HF tokenizer Building Vocab

Building Vocab

```
vocab = tokenizer.get_vocab()  
print(vocab)
```

```
{'like': 8, 'expensive': 6, 'i': 7, 'are': 3, '<pad>': 1, 'a': 2, 'books': 4, 'dog': 5, '<unk>': 0}
```

▶ # Initialize the tokenizer and define a trainer
tokenizer = Tokenizer(WordLevel())
tokenizer.pre_tokenizer = Whitespace()

Initialize
Tokenizer

Initialize
Tokenization

```
corpus = [  
    "i like a dog",  
    "books are expensive"  
]
```

```
['i', 'like', 'a', 'dog']  
['books', 'are', 'expensive', '<pad>']
```

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary

HF tokenizer Building Vocab

Building Vocab

```
vocab = tokenizer.get_vocab()  
print(vocab)
```

```
{'like': 8, 'expensive': 6, 'i': 7, 'are': 3, '<pad>': 1, 'a': 2, 'books': 4, 'dog': 5, '<unk>': 0}
```

Handle
Padding&Truncation

Define vocab_size,
special_tokens

Build Vocab From
Corpus

```
▶ tokenizer.enable_padding(pad_id=1,  
                           pad_token="<pad>",  
                           length=sequence_length)  
tokenizer.enable_truncation(max_length=sequence_length)
```

```
▶ # Train the tokenizer on your corpus  
trainer = WordLevelTrainer(vocab_size=vocab_size,  
                           special_tokens=["<unk>", "<pad>"])  
tokenizer.train_from_iterator(corpus, trainer)
```

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

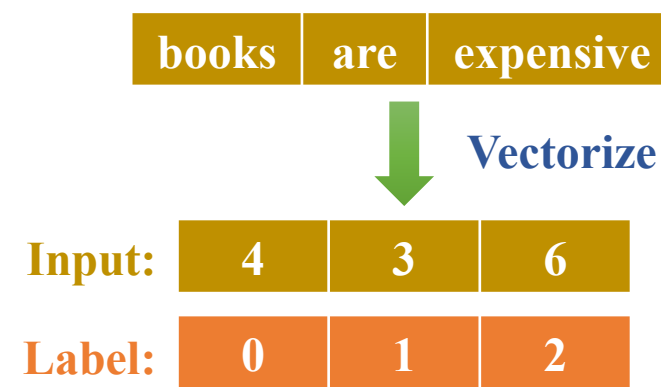
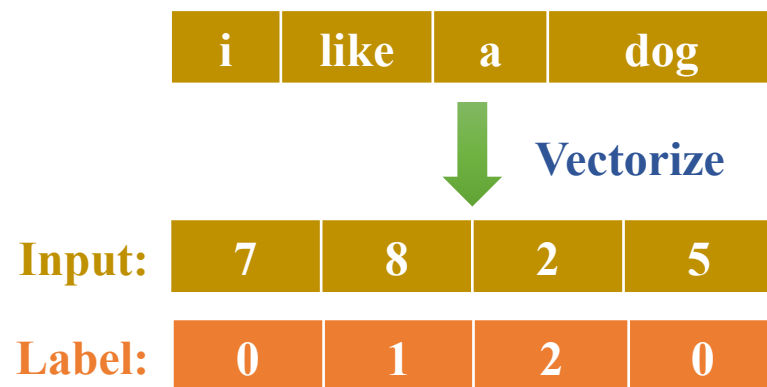
Vocabulary

HF tokenizer Building Vocab

Vectorization

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

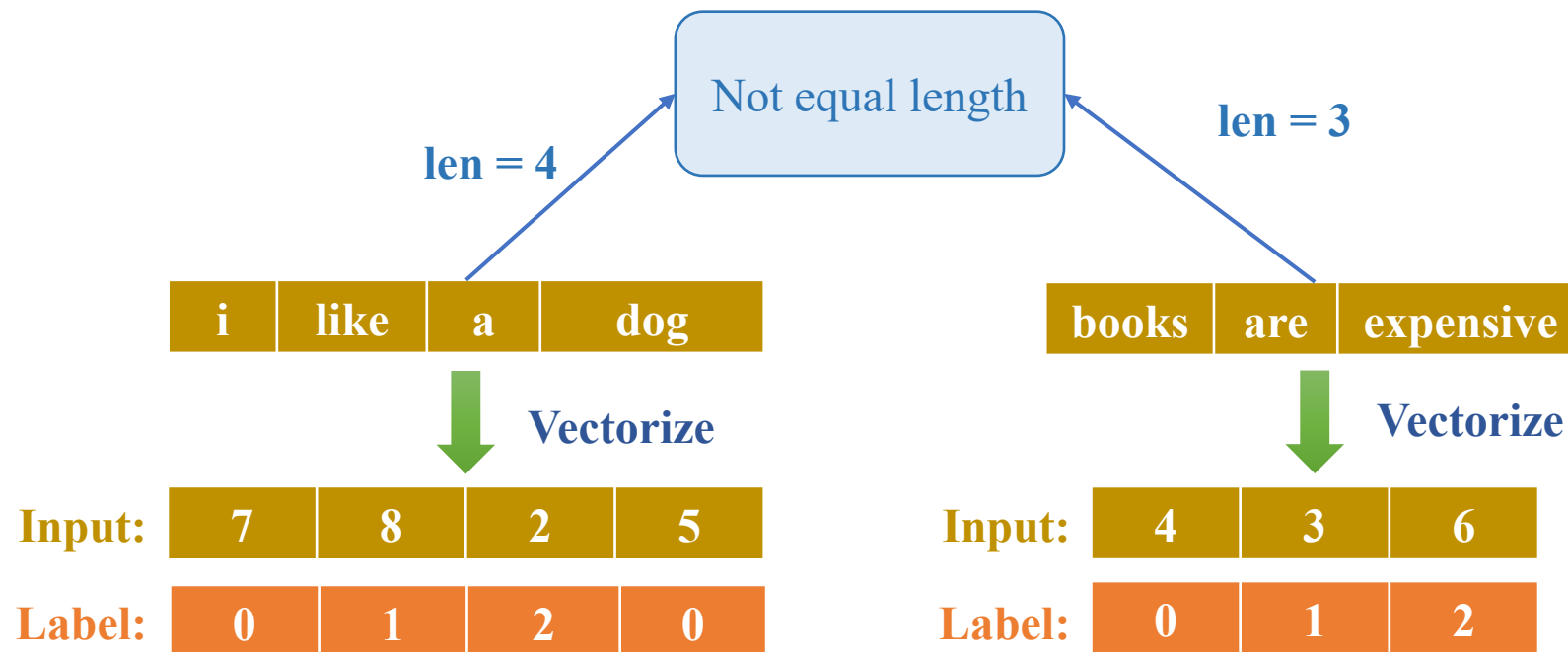
Vocabulary



Vectorization

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary

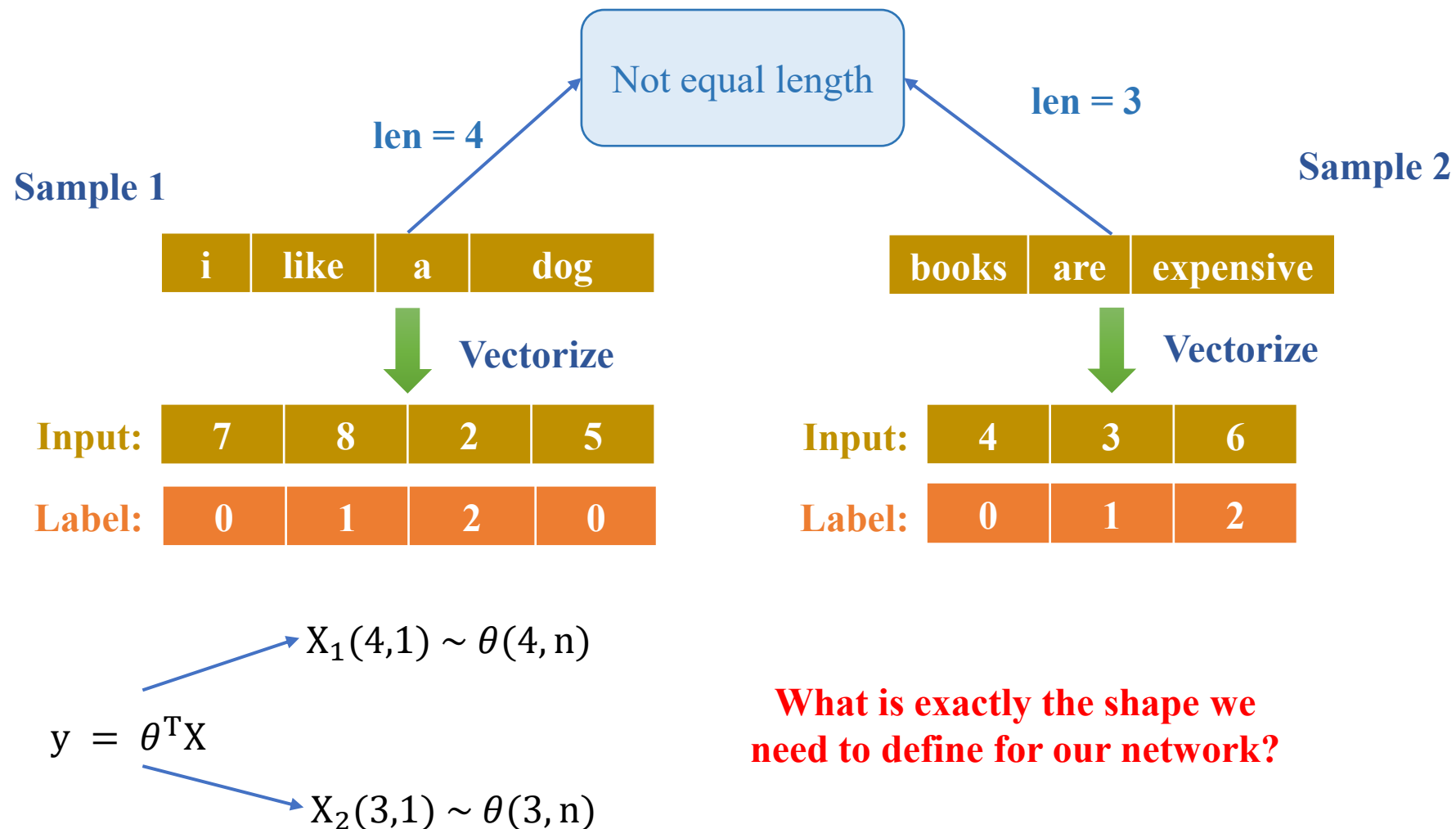


What is a problem?

Vectorization

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary



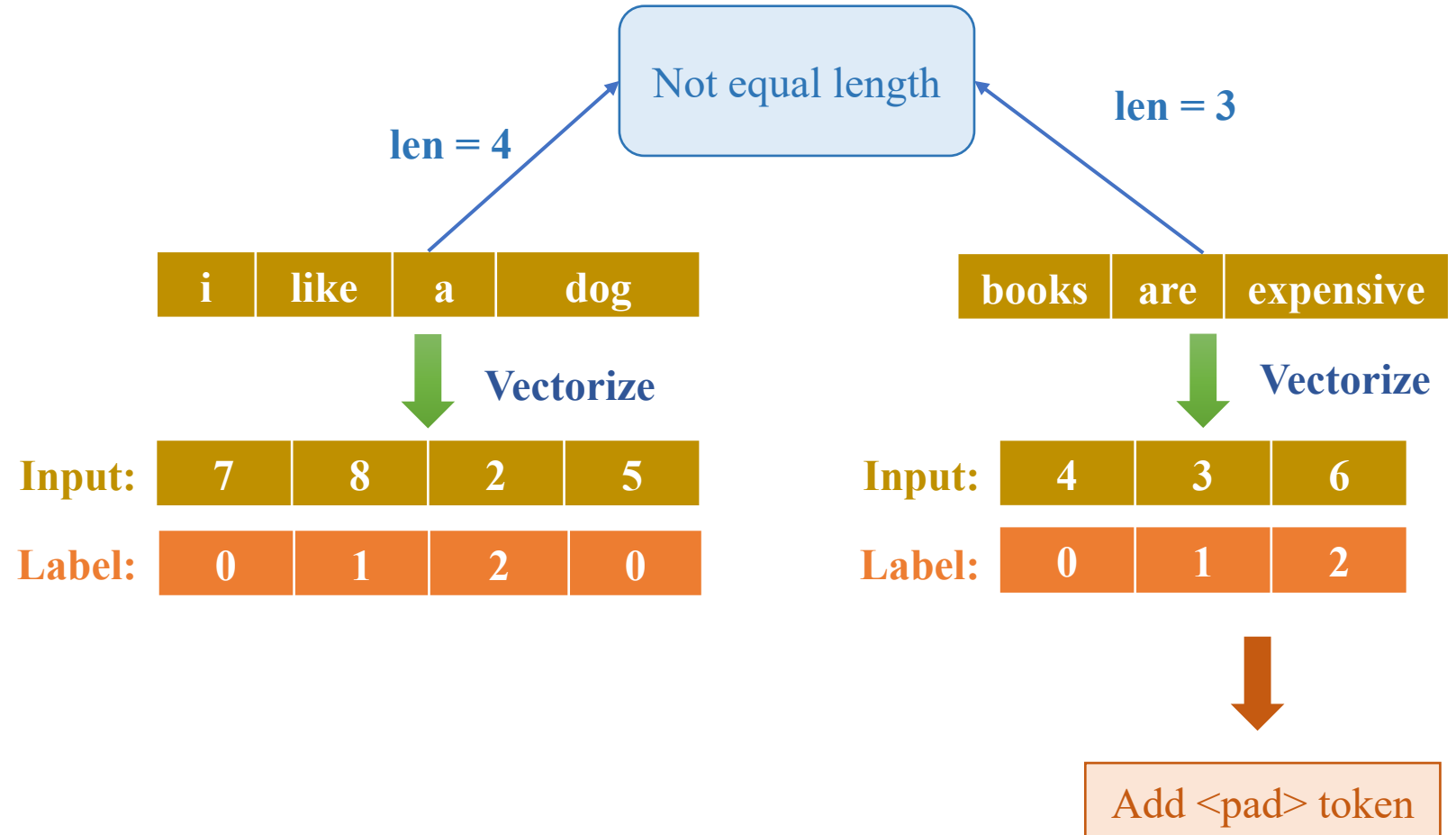
What is exactly the shape we need to define for our network?

Vectorization

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary

sequence_length = 4

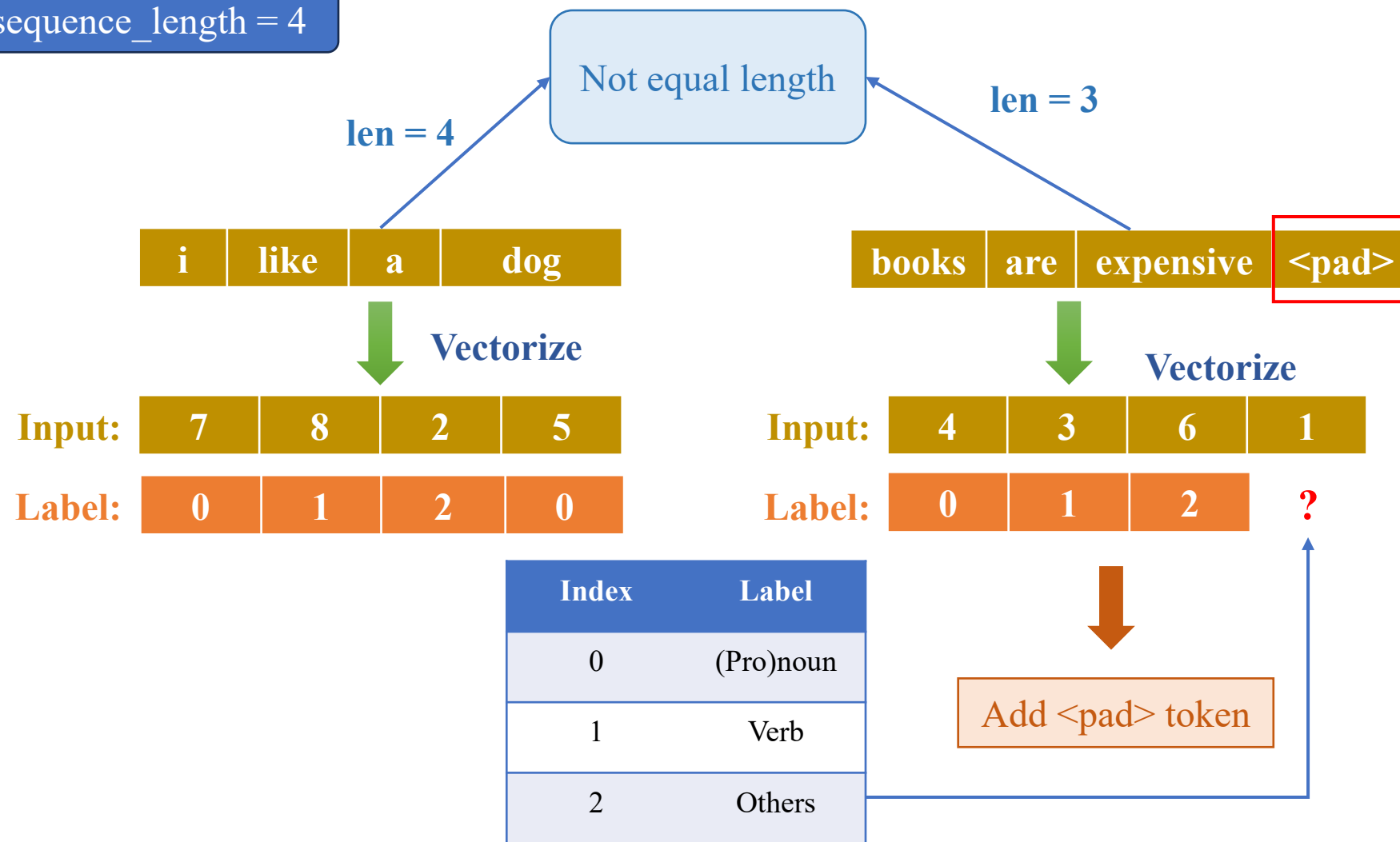


Vectorization

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary

sequence_length = 4

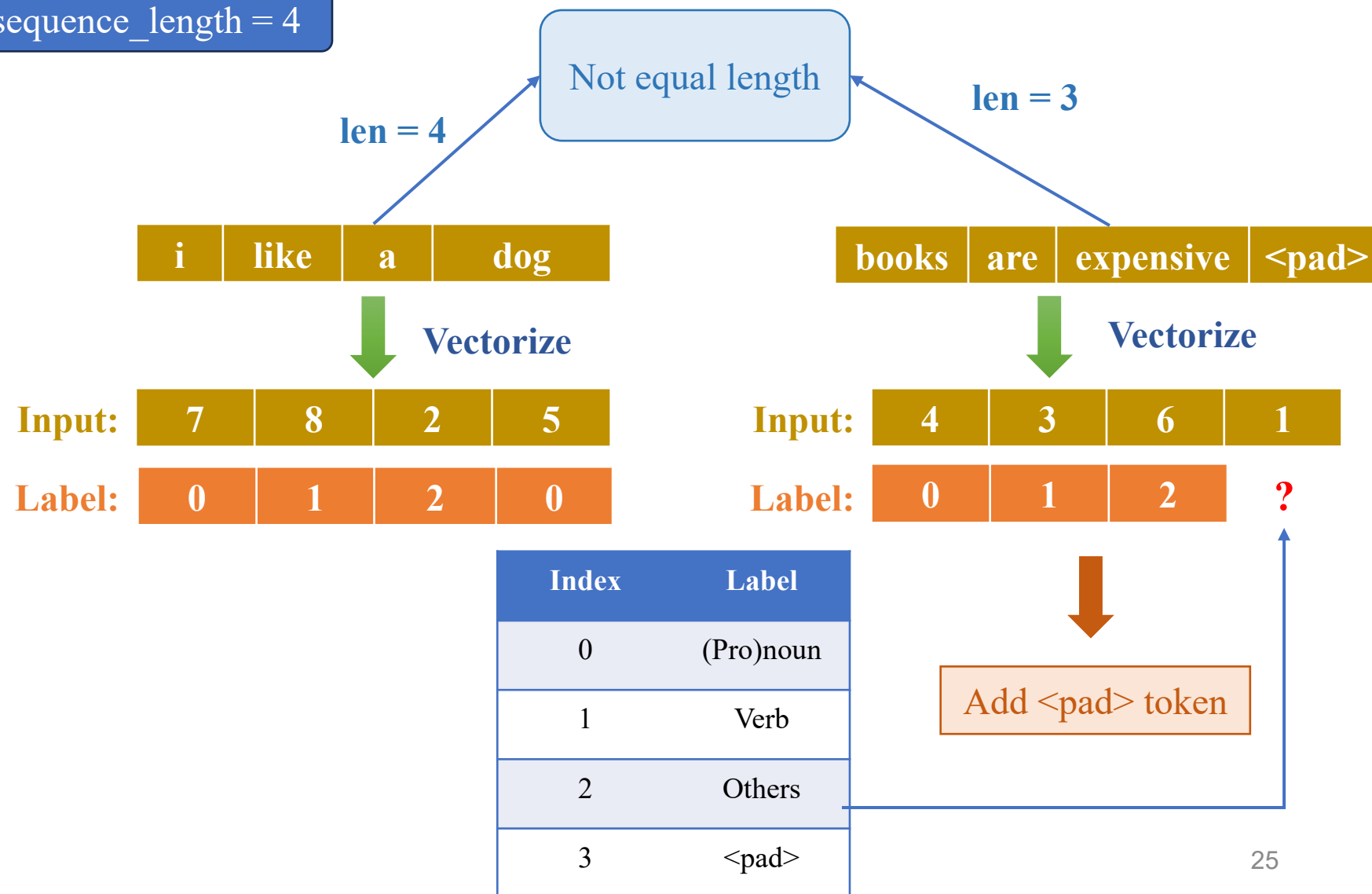


Vectorization

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary

sequence_length = 4

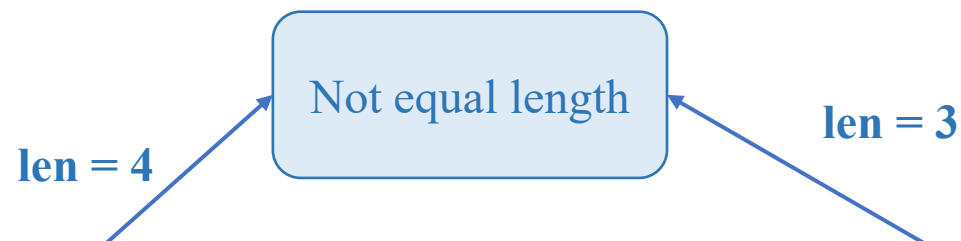


Vectorization

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary

sequence_length = 4



```
# Tokenize samples
def vectorize_text(sentence, tokenizer):
    output = tokenizer.encode(sentence)
    return torch.tensor(output.ids, dtype=torch.long)

# Vectorize samples
def vectorize_label(sentence_labels, sequence_length):
    sentence_labels = sentence_labels + [3]*(sequence_length - len(sentence_labels))
    return torch.tensor(sentence_labels, dtype=torch.long)
```

expensive <pad>

Vectorize

6	1
2	3

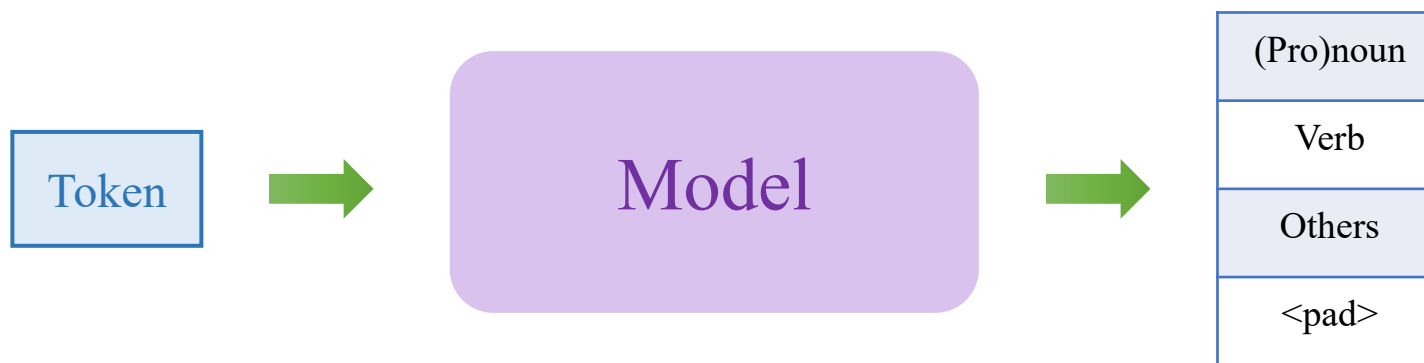
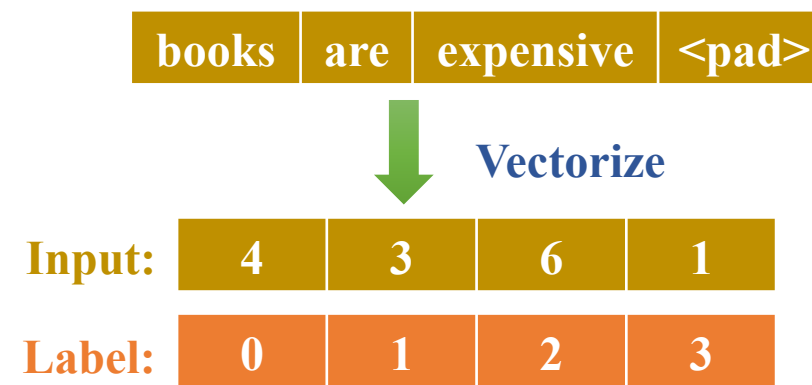
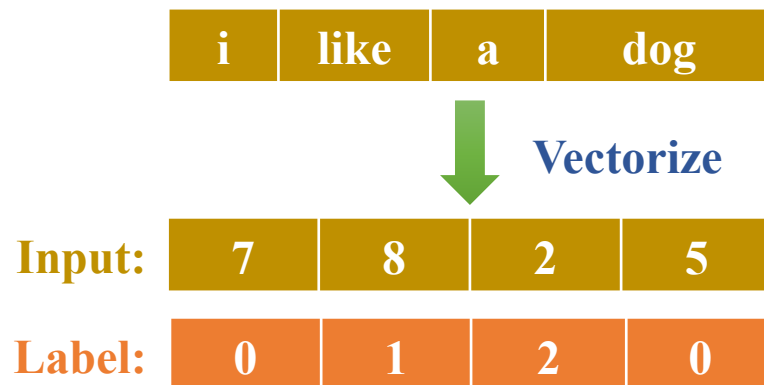
Add <pad> token

Vectorization

0	<unk>
1	<pad>
2	a
3	are
4	books
5	dog
6	expensive
7	i
8	like

Vocabulary

sequence_length = 4



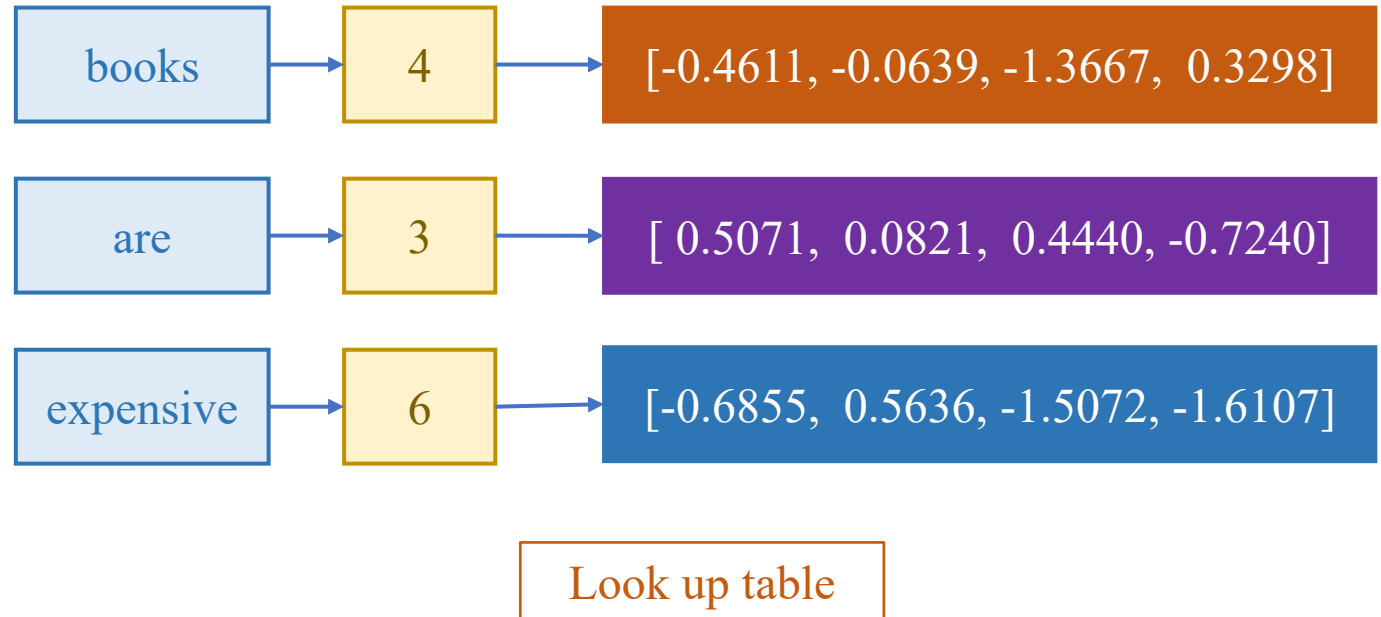
→ Token classification problem

Part-of-Speech Tagging

❖ Embedding

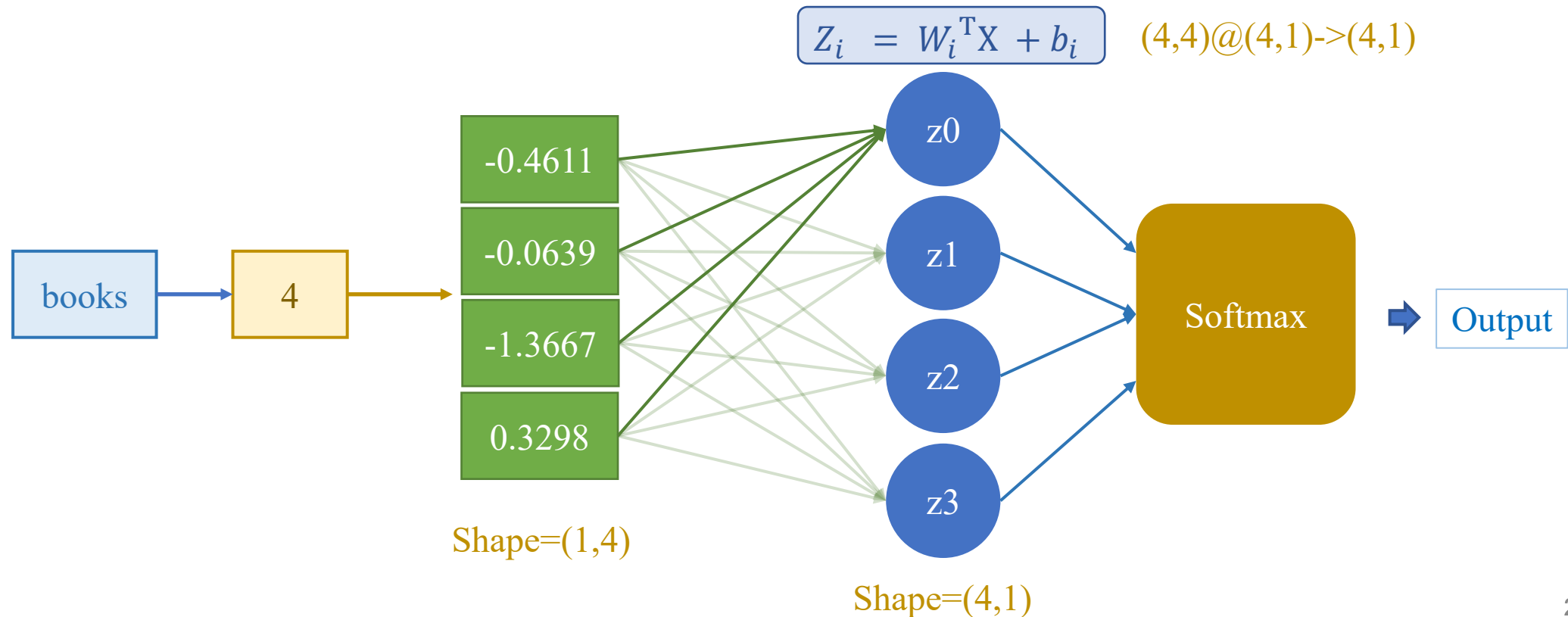
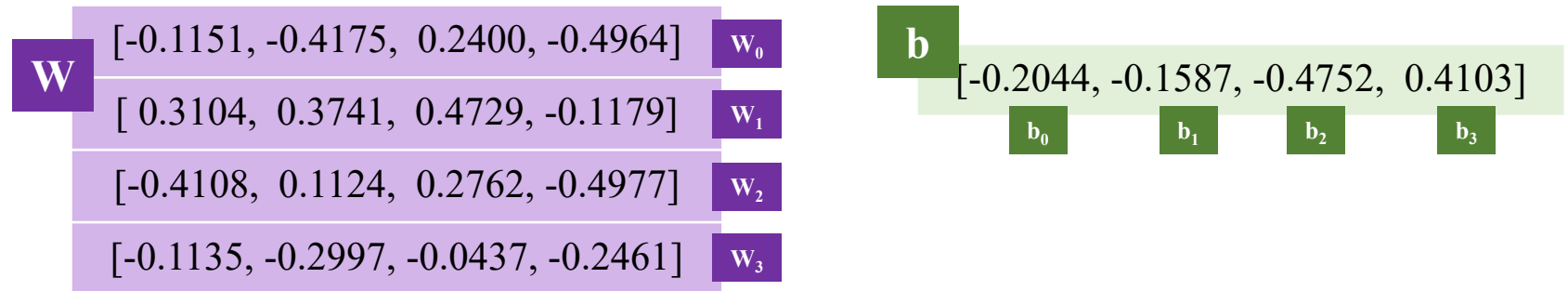
index	vocab	Embedding vector
0	<unk>	[1.3032, 0.4879, 1.1340, -0.3556]
1	<pad>	[0.3618, 1.9993, 0.6630, 0.7047]
2	a	[0.0213, -0.8293, -1.0809, -0.7839]
3	are	[0.5071, 0.0821, 0.4440, -0.7240]
4	books	[-0.4611, -0.0639, -1.3667, 0.3298]
5	dog	[0.6427, 0.5742, 0.5867, -0.0188]
6	expensive	[-0.6855, 0.5636, -1.5072, -1.6107]
7	i	[-0.8051, 0.5158, -0.7129, 0.2196]
8	like	[-1.5988, -0.1091, 0.7152, 0.0391]

From Word2Vec models or
learnable parameters



Part-of-Speech Tagging

❖ A word input



Part-of-Speech Tagging

❖ A word input

$$Z_0 = XW_0 + b_0$$

X

-0.4611
-0.0639
-1.3667
0.3298

x

W₀

[-0.1151, -0.4175, 0.2400, -0.4964]

+

b₀

0.1987

0.0531
0.0267
-0.3280
-0.1637

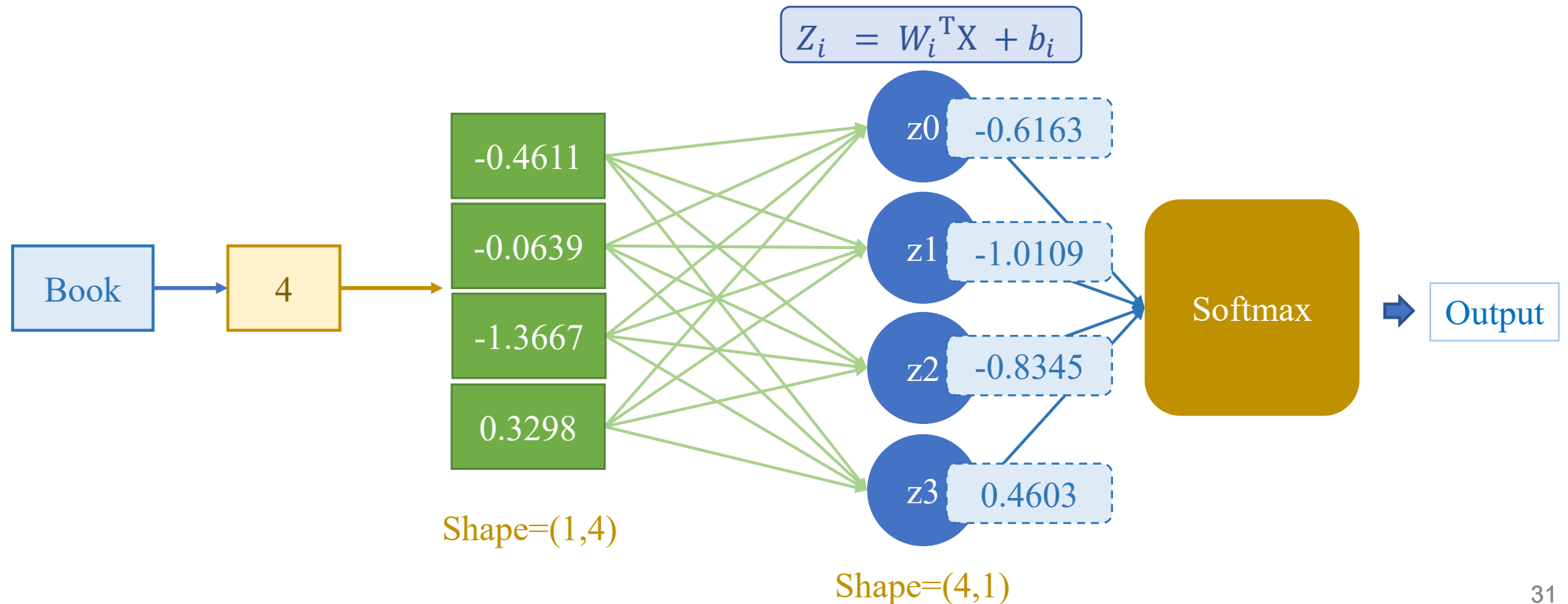
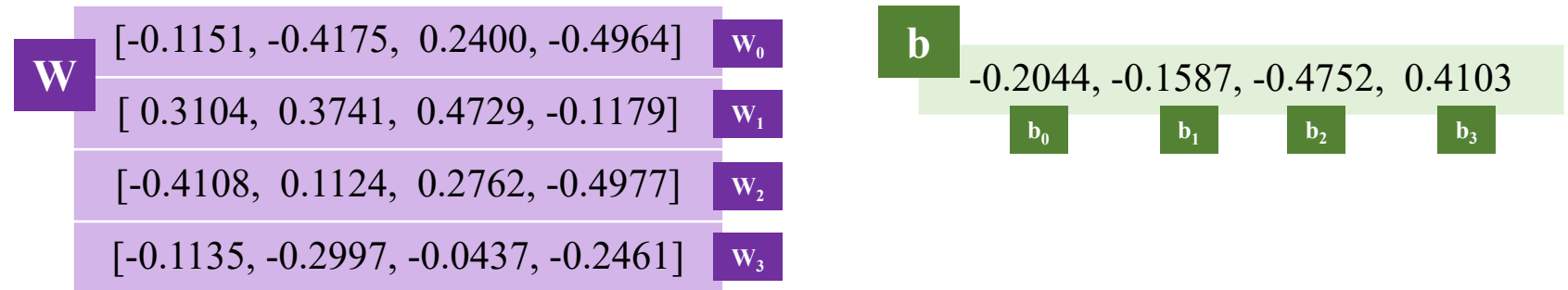
$$\begin{aligned} \text{Sum} &= -0.4120 + -0.2044 = Z_0 \\ XW_0 & \quad b_0 & = & \quad -0.6163 \end{aligned}$$

W	[-0.1151, -0.4175, 0.2400, -0.4964]	w₀
	[0.3104, 0.3741, 0.4729, -0.1179]	w₁
	[-0.4108, 0.1124, 0.2762, -0.4977]	w₂
	[-0.1135, -0.2997, -0.0437, -0.2461]	w₃

b	[-0.2044, -0.1587, -0.4752, 0.4103]
	b₀ b₁ b₂ b₃

Part-of-Speech Tagging

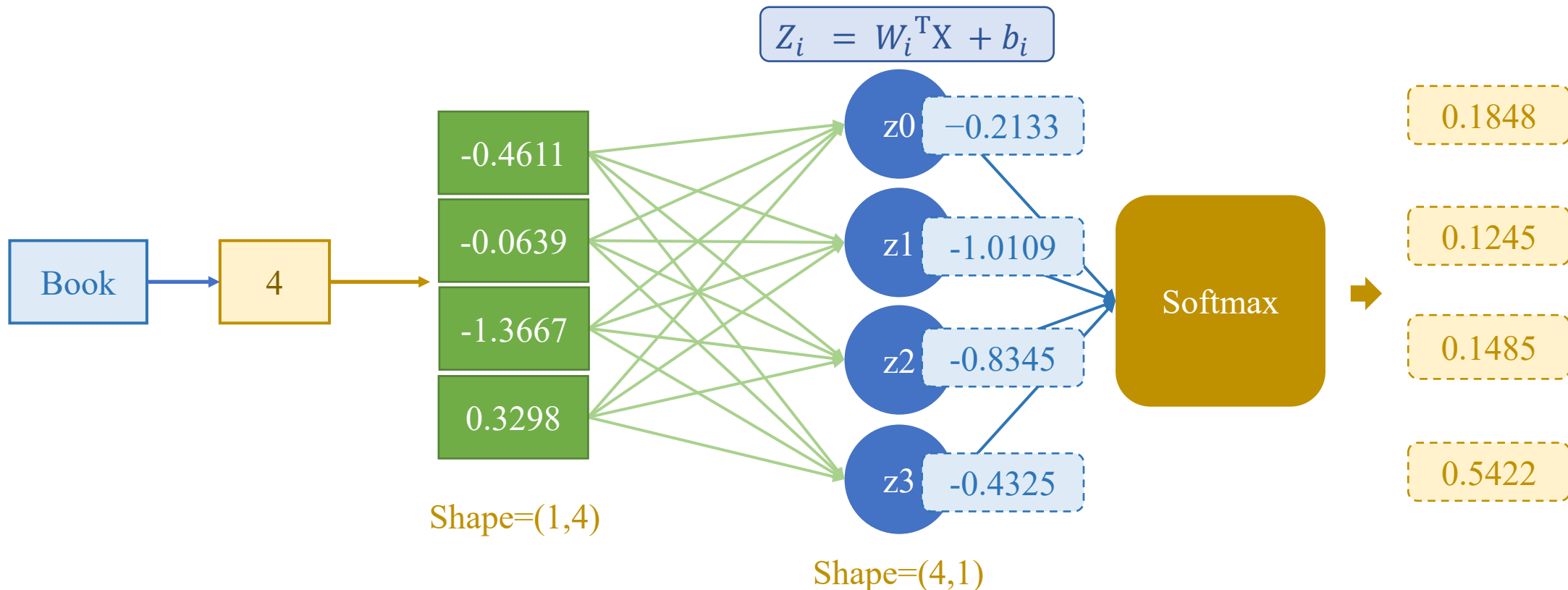
❖ A word input



Part-of-Speech Tagging

❖ A word input

W	[-0.1151, -0.4175, 0.2400, -0.4964]	w ₀	b	-0.2044, -0.1587, -0.4752, 0.4103			
	[0.3104, 0.3741, 0.4729, -0.1179]	w ₁		b ₀	b ₁	b ₂	b ₃
	[-0.4108, 0.1124, 0.2762, -0.4977]	w ₂					
	[-0.1135, -0.2997, -0.0437, -0.2461]	w ₃					



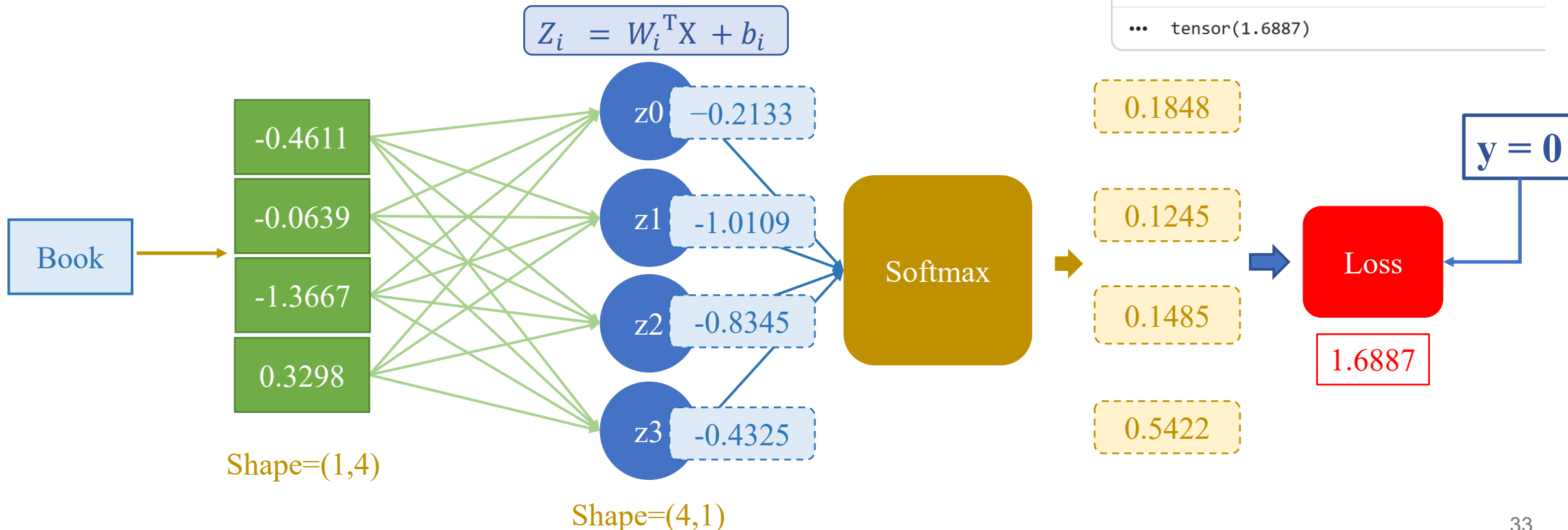
Part-of-Speech Tagging

❖ A word input

W	[-0.1151, -0.4175, 0.2400, -0.4964]	w_0
	[0.3104, 0.3741, 0.4729, -0.1179]	w_1
	[-0.4108, 0.1124, 0.2762, -0.4977]	w_2
	[-0.1135, -0.2997, -0.0437, -0.2461]	w_3

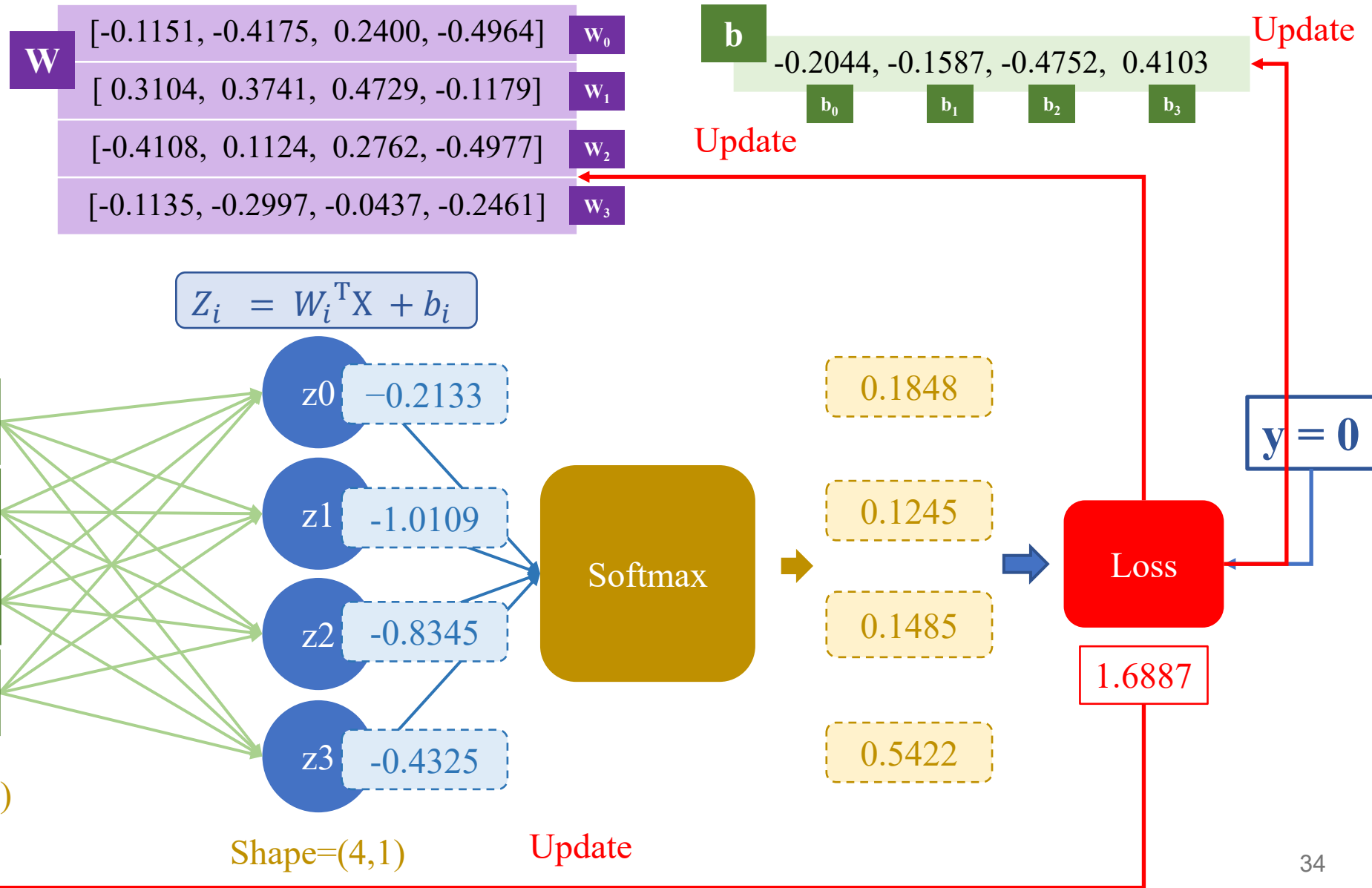
b	-0.2044	-0.1587	-0.4752	0.4103
	b_0	b_1	b_2	b_3

```
loss = -torch.log(softmax_output[0])  
loss  
... tensor(1.6887)
```



Part-of-Speech Tagging

❖ A word input



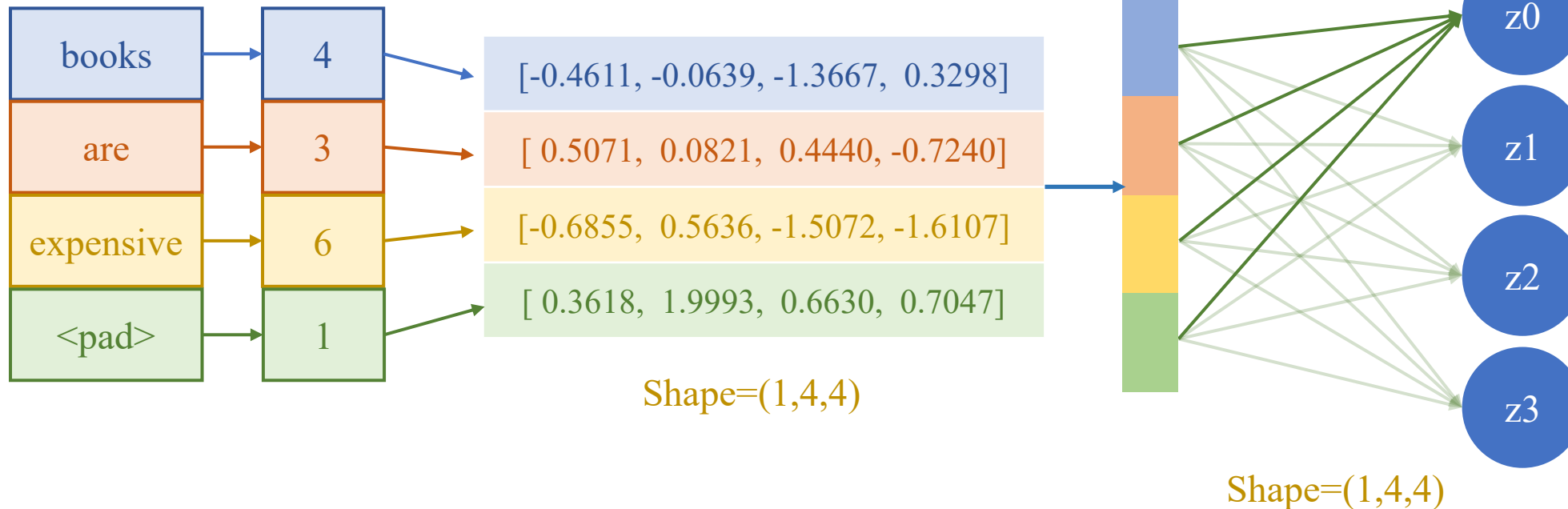
Part-of-Speech Tagging

❖ A word input

W	[-0.1151, -0.4175, 0.2400, -0.4964]	w ₀
	[0.3104, 0.3741, 0.4729, -0.1179]	w ₁
	[-0.4108, 0.1124, 0.2762, -0.4977]	w ₂
	[-0.1135, -0.2997, -0.0437, -0.2461]	w ₃

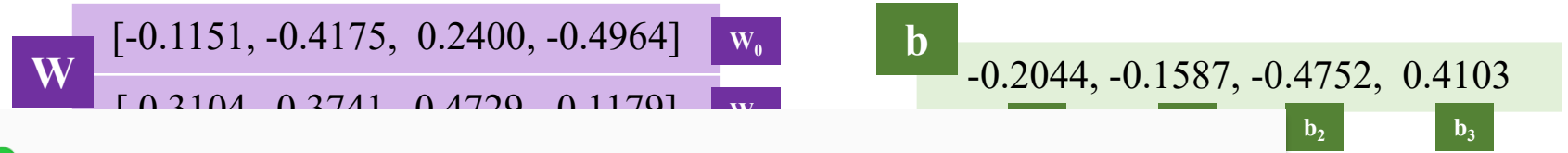
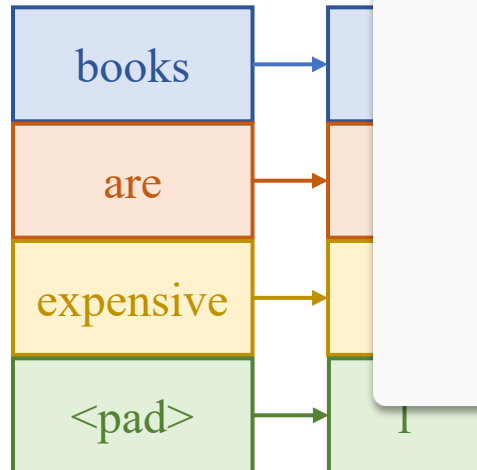
b	-0.2044, -0.1587, -0.4752, 0.4103			
	b ₀	b ₁	b ₂	b ₃

$$Z_i = W_i^T X + b_i$$



Part-of-Speech Tagging

❖ A word input



```
class POS_Model(nn.Module):
    def __init__(self, vocab_size, num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, 4)
        self.fc = nn.Linear(4, num_classes)

    def forward(self, x):
        x = self.embedding(x)
        x = self.fc(x)
        return x.permute(0, 2, 1)
```

Shape=(1,4,4)

Shape=(1,4,4)

$$TX + b_i$$

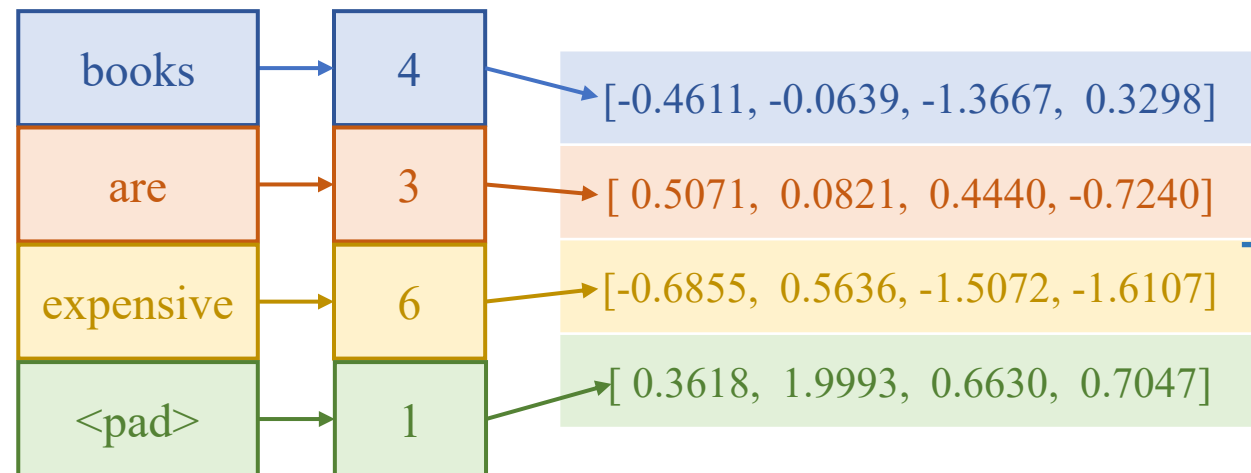
z3

Part-of-Speech Tagging

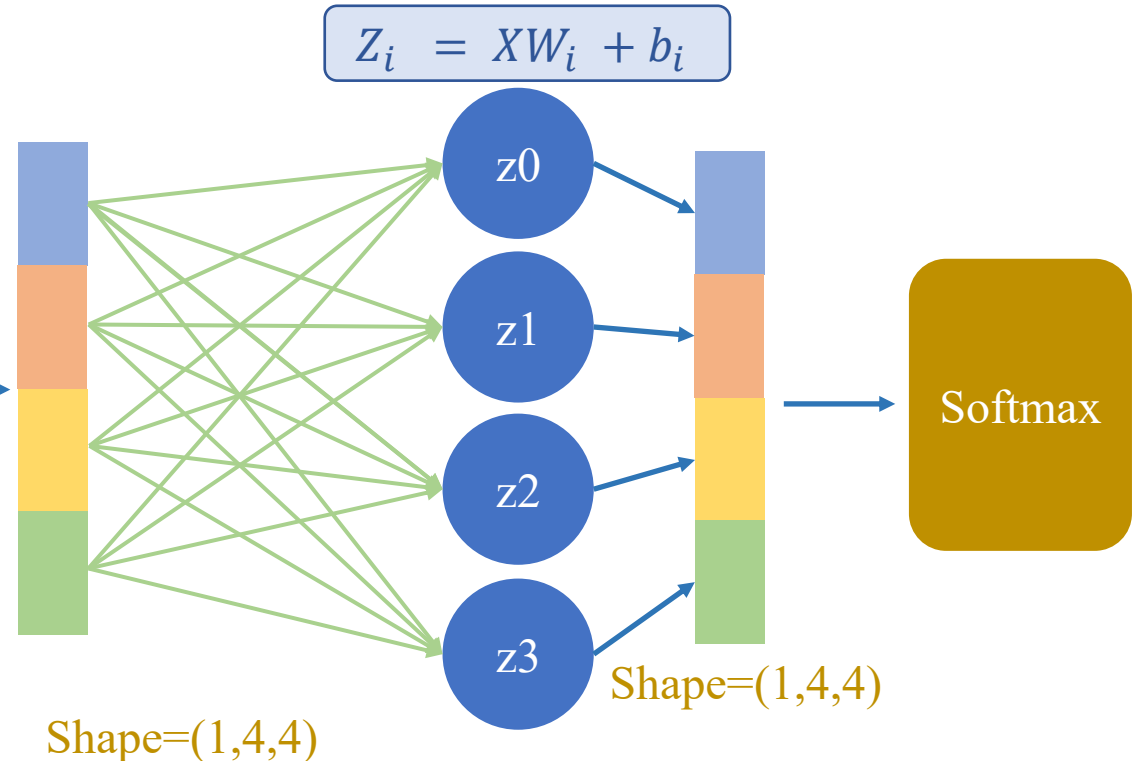
❖ A word input

W	[-0.1151, -0.4175, 0.2400, -0.4964]	w ₀
	[0.3104, 0.3741, 0.4729, -0.1179]	w ₁
	[-0.4108, 0.1124, 0.2762, -0.4977]	w ₂
	[-0.1135, -0.2997, -0.0437, -0.2461]	w ₃

b	-0.2044, -0.1587, -0.4752, 0.4103			
	b ₀	b ₁	b ₂	b ₃



Shape=(1,4,4)



Why permute?

Part-of-Speech Tagging

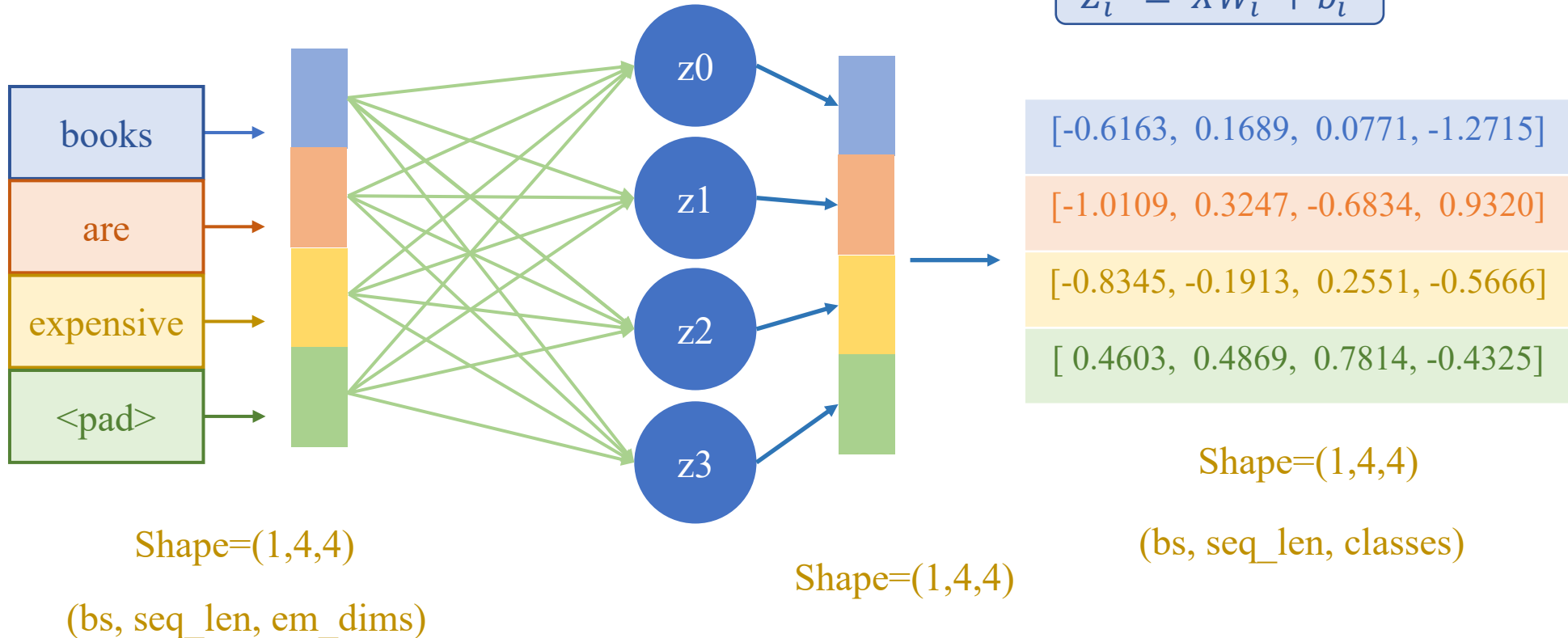
❖ A word input

nn.Linear

W	[-0.1151, -0.4175, 0.2400, -0.4964]	w₀
	[0.3104, 0.3741, 0.4729, -0.1179]	w₁
	[-0.4108, 0.1124, 0.2762, -0.4977]	w₂
	[-0.1135, -0.2997, -0.0437, -0.2461]	w₃

b	-0.2044, -0.1587, -0.4752, 0.4103			
	b₀	b₁	b₂	b₃

$$Z_i = XW_i + b_i$$



Part-of-Speech Tagging

❖ A word input

nn.Linear

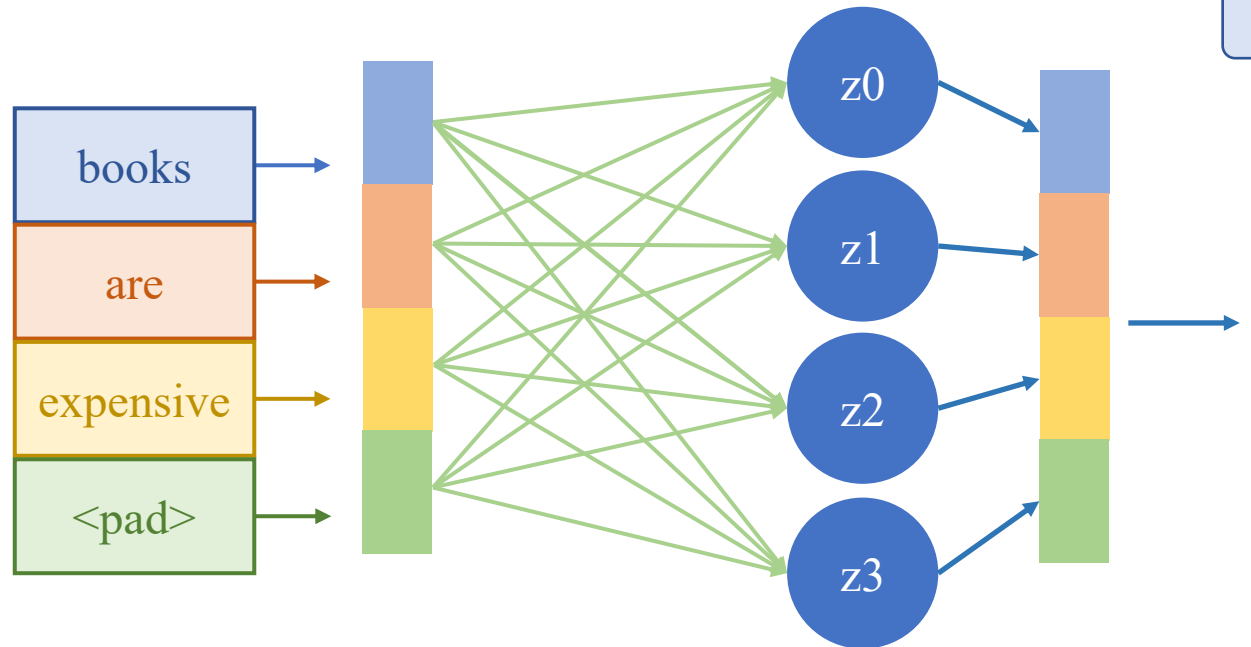
W	$[-0.1151, -0.4175, 0.2400, -0.4964]$	w₀
	$[0.3104, 0.3741, 0.4729, -0.1179]$	w₁
	$[-0.4108, 0.1124, 0.2762, -0.4977]$	w₂
	$[-0.1135, -0.2997, -0.0437, -0.2461]$	w₃

b	$[-0.2044, -0.1587, -0.4752, 0.4103]$
	b₀ b₁ b₂ b₃

Input shape: (N, C, d)
Target shape: (N, d)

Pytorch

nn.CrossEntropyLoss



$$Z_i = XW_i + b_i$$

Shape=(1,4,4)

(bs, seq_len, em_dims)

Shape=(1,4,4)

Shape=(1,4,4)

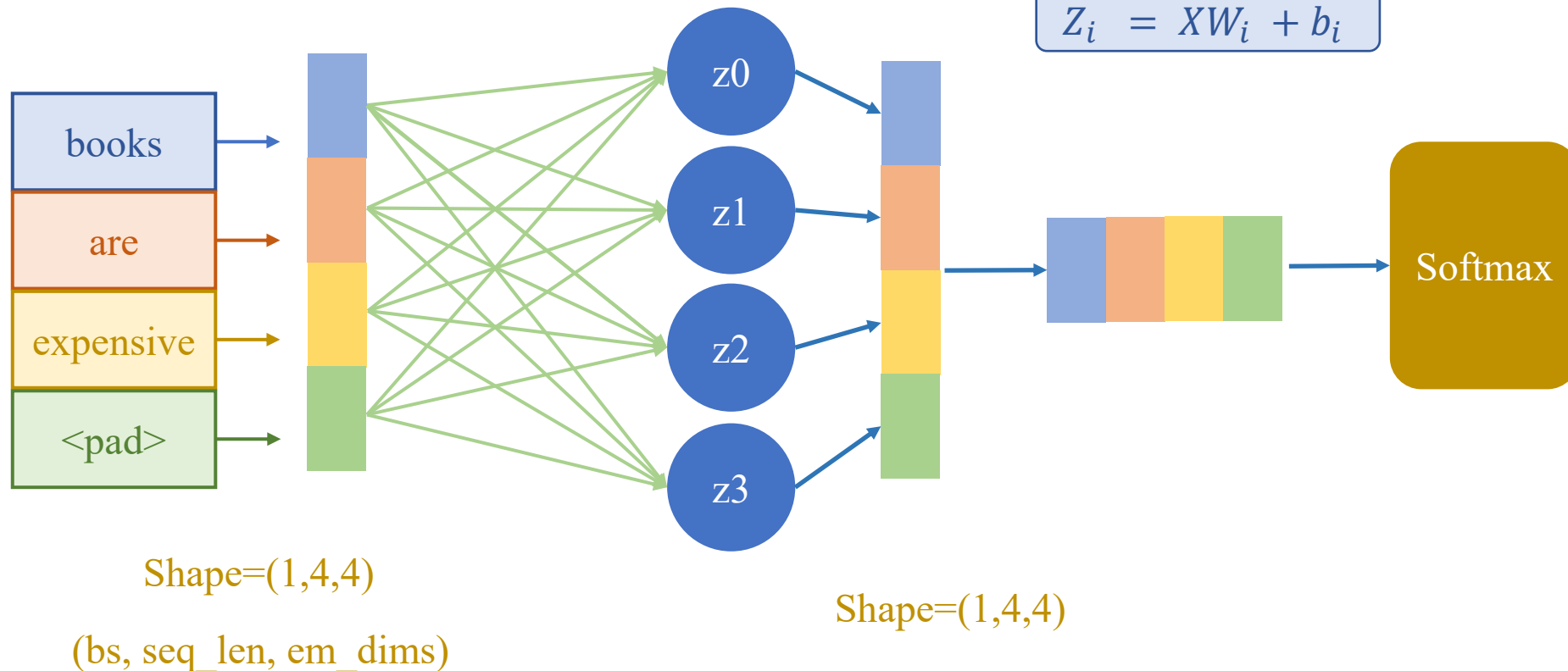
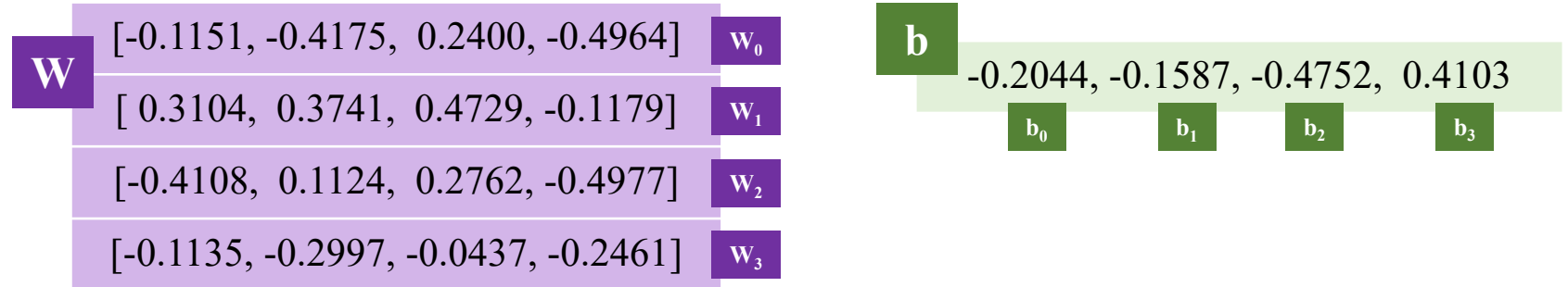
(bs, seq_len, classes)

permute

Part-of-Speech Tagging

❖ A word input

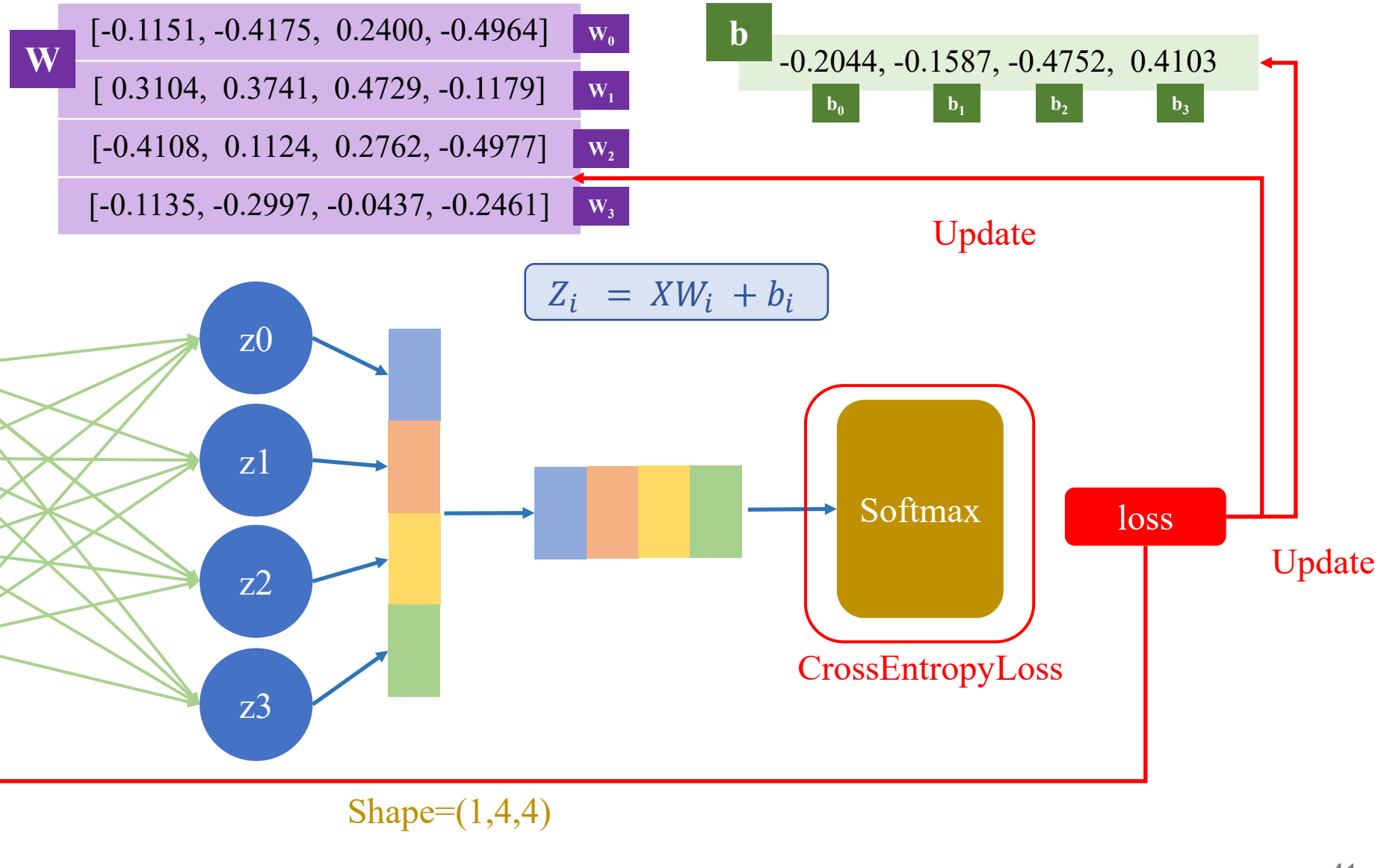
nn.Linear



Part-of-Speech Tagging

❖ A word input

nn.Linear



Named-entity recognition (NER)

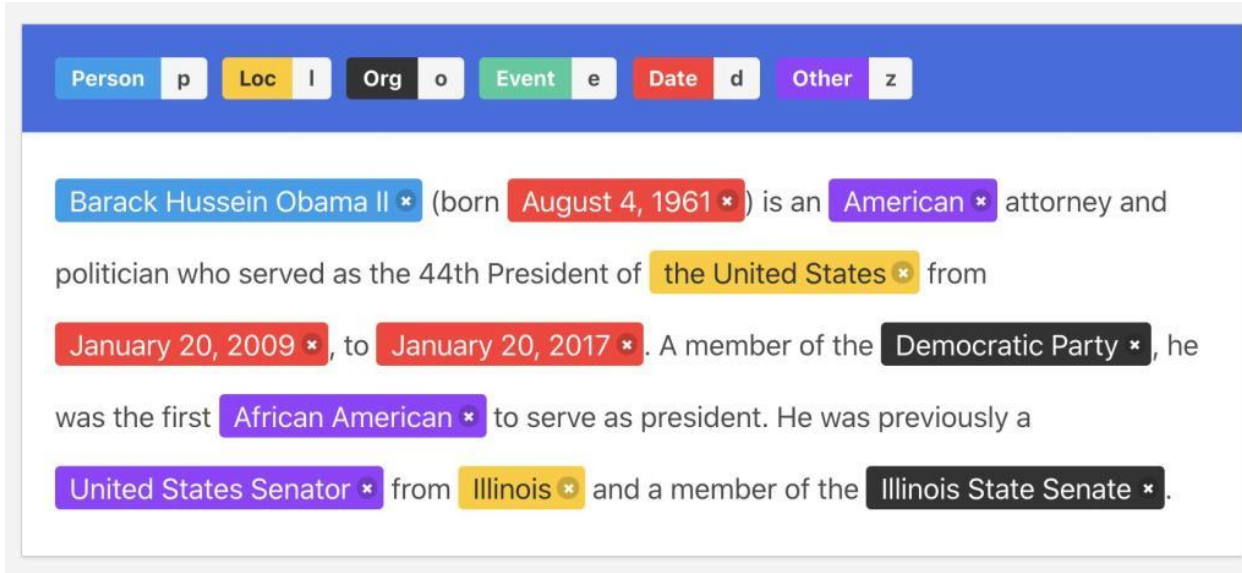
Named-entity recognition (NER)

- Introduction
- Preprocessing
- Embedding
- Named-entity recognition (NER)

Named Entity Recognition (NER)

❖ Introduction

Named Entity Recognition (NER) is a fundamental task in **Natural Language Processing (NLP)** that identifies and classifies named entities in text.



The screenshot displays a text snippet with several named entities highlighted in colored boxes and labeled with a letter in a small box. The labels are: Person (p), Loc (l), Org (o), Event (e), Date (d), and Other (z). The text is: "Barack Hussein Obama II * (born August 4, 1961 *) is an American * attorney and politician who served as the 44th President of the United States * from January 20, 2009 *, to January 20, 2017 *. A member of the Democratic Party *, he was the first African American * to serve as president. He was previously a United States Senator * from Illinois * and a member of the Illinois State Senate *." The labels are: Person (p), Loc (l), Org (o), Event (e), Date (d), and Other (z).

Common entity categories include:

- **PER (Person):** Names of people.
- **ORG (Organization):** Company or institution names.
- **LOC (Location):** Geographic locations.
- **DATE (Date & Time):** Specific dates, years, and time expressions.
- **EVENT (Events & Festivals):** Named events.
- ...

Named Entity Recognition (NER)

❖ Applications



Information Retrieval
& Search Engines



Content Recommendation
Systems



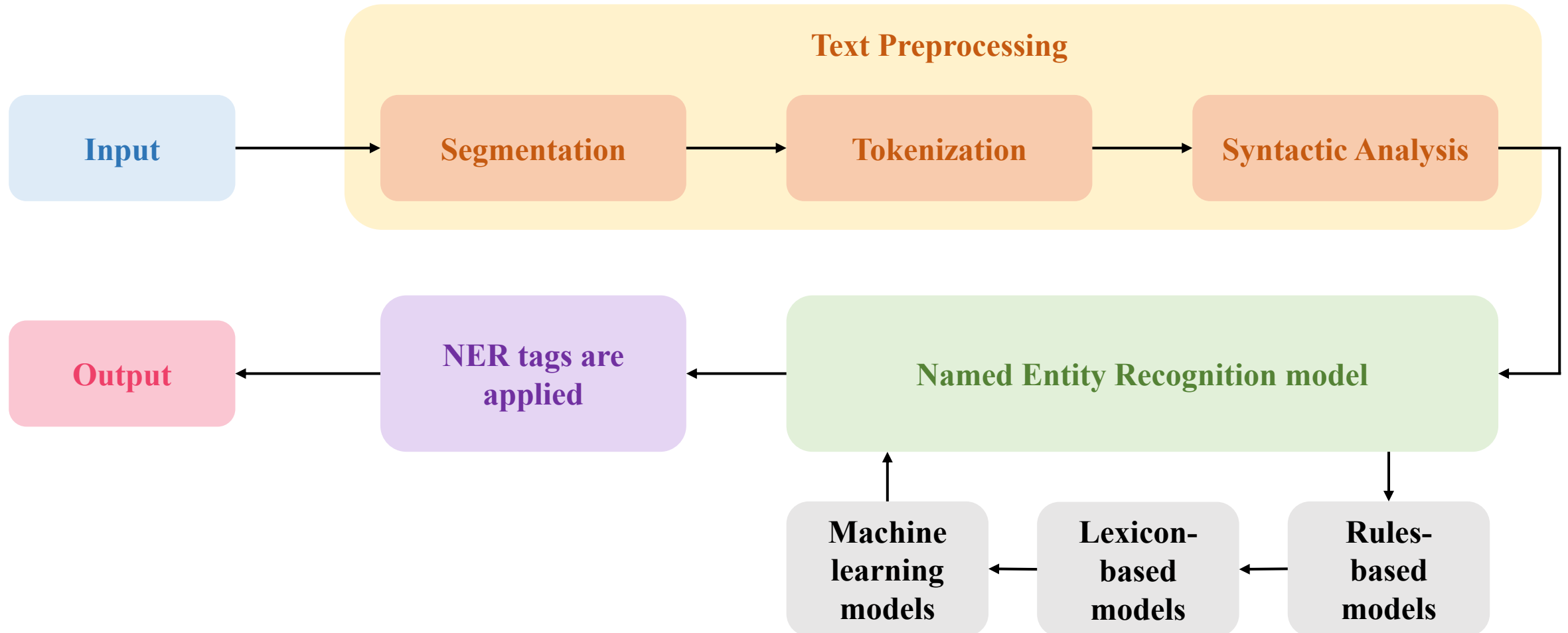
Content Classification



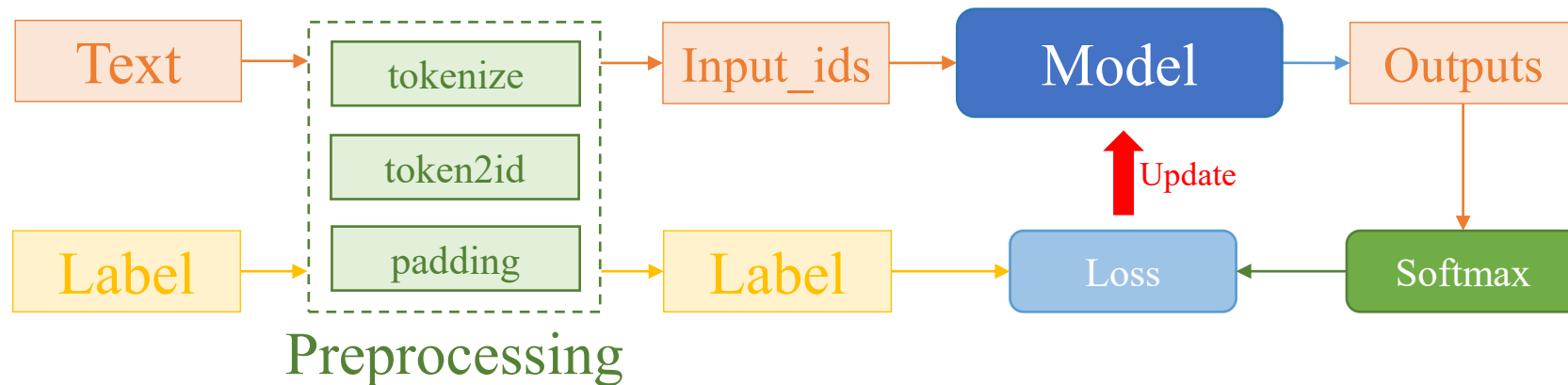
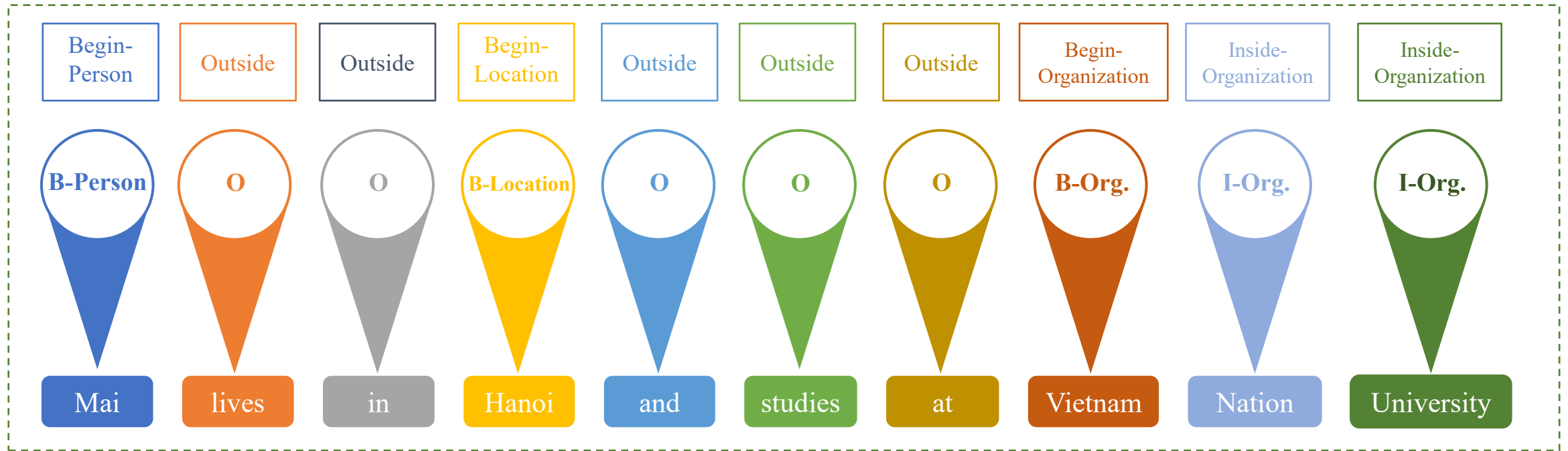
Chatbots &
Virtual Assistants

Named Entity Recognition (NER)

❖ Preprocessing steps



Named Entity Recognition (NER)

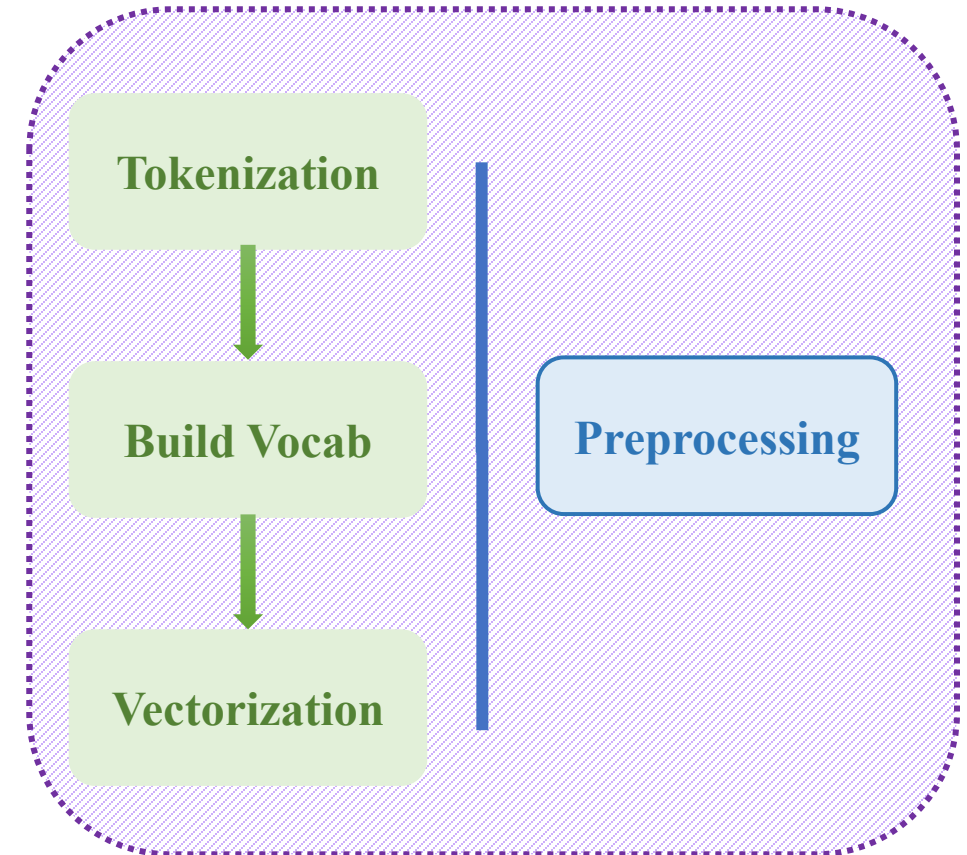


Courpus

❖ Tokenization

```
corpus = [  
    "sundar pichai is CEO of google",  
    "elon musk buy twitter"  
]
```

```
labels = [[0, 1, 4, 4, 4, 2],  
          [0, 1, 4, 2]]
```



Label

0	1	2	3	4
B-Person	I-Person	B-Organization/Location	I-Organization/Location	O

Preprocessing

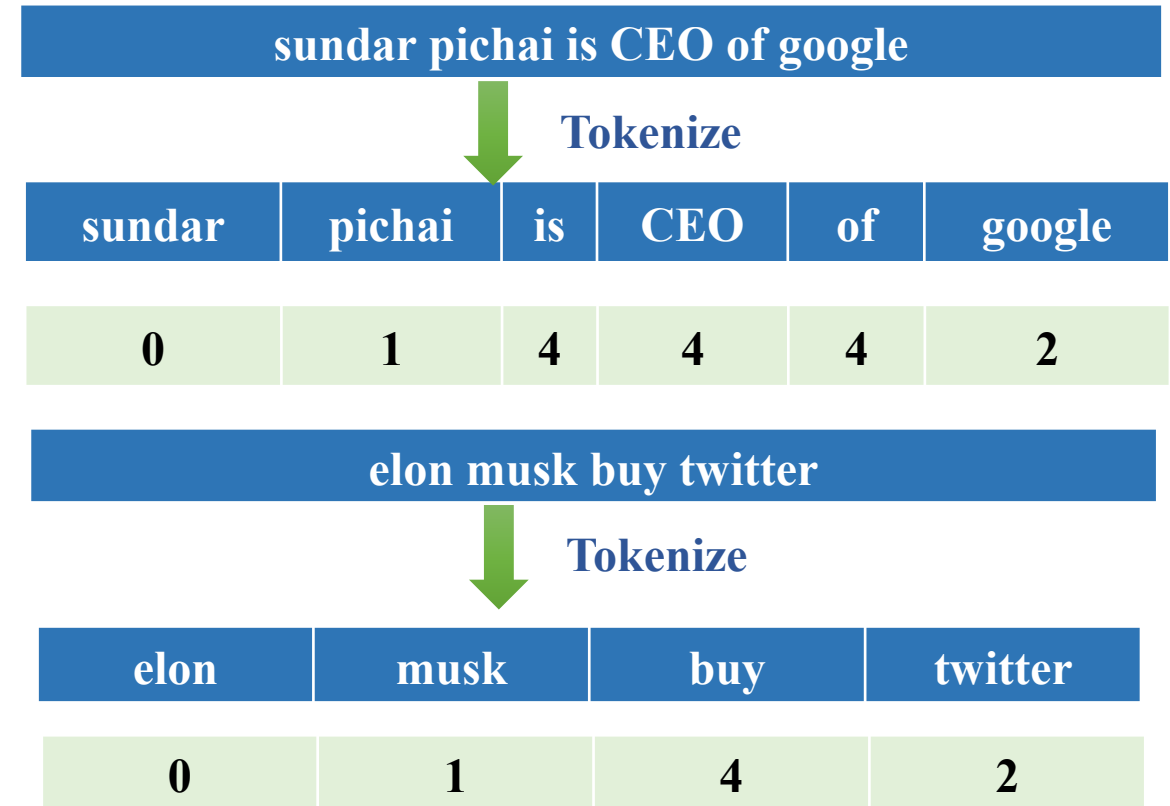
❖ Tokenization

```
corpus = [
    "sundar pichai is CEO of google",
    "elon musk buy twitter"
]
```

```
labels = [[0, 1, 4, 4, 4, 2],
          [0, 1, 4, 2]]
```

Label

0	1	2	3	4
B-Person	I-Person	B-Organization/Location	I-Organization/Location	O



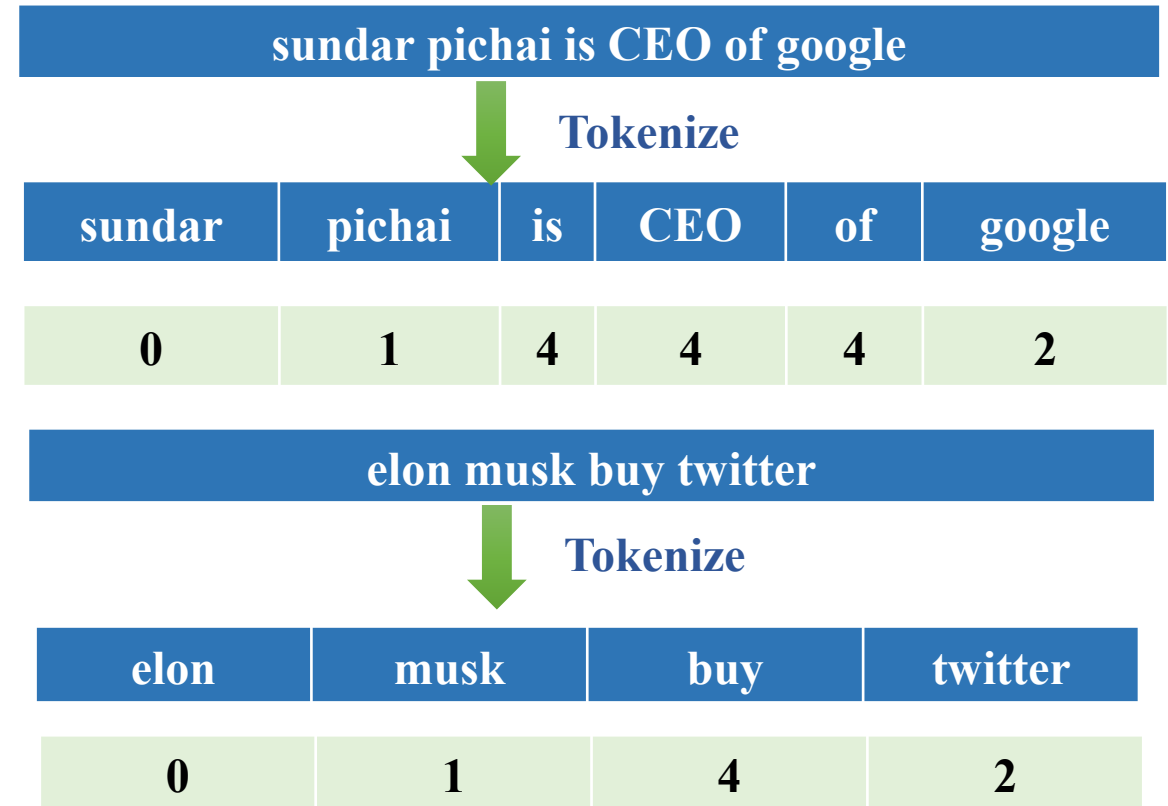
Preprocessing

❖ Tokenization

```
corpus = [  
    "sundar pichai is CEO of google",  
    "elon musk buy twitter"  
]
```

```
▶ sample1 = corpus[0]  
   sample2 = corpus[1]  
  
   sample1_token = tokenizer.encode(sample1)  
   print(sample1_token.tokens)  
  
   sample2_token = tokenizer.encode(sample2)  
   print(sample2_token.tokens)
```

```
... ['sundar', 'pichai', 'is', 'CEO', 'of', 'google']  
    ['elon', 'musk', 'buy', 'twitter', '<pad>', '<pad>']
```



Preprocessing

❖ Building vocab

```
▶ # Initialize the tokenizer and define a trainer
tokenizer = Tokenizer(WordLevel())
tokenizer.pre_tokenizer = Whitespace()
tokenizer.enable_padding(pad_id=1,
                        pad_token="<pad>",
                        length=sequence_length)
tokenizer.enable_truncation(max_length=sequence_length)
# Train the tokenizer on your corpus
trainer = WordLevelTrainer(vocab_size=vocab_size,
                           special_tokens=["<unk>", "<pad>"])
tokenizer.train_from_iterator(corpus, trainer)
```

Vocabulary
vocab_size = 12

0	<unk>
1	<pad>
2	buy
3	ceo
4	elon
5	google
6	is
7	musk
8	of
9	pichai
10	sundar
11	twitter

Preprocessing

sundar pichai is CEO of google

↓ Tokenize

sundar pichai is CEO of google

↓ Vectorize

10 9 6 3 8 5

0 1 4 4 4 2

elon musk buy twitter

↓ Tokenize

elon musk buy twitter

↓ Vectorize

4 7 2 11 1 1

0 1 4 2 5 5

0	1	2	3	4	5
B-Person	I-Person	B-Organization/Location	I-Organization/Location	O	<pad>

0	1	2	3	4	5	6	7	8	9	10	11
<unk>	<pad>	buy	ceo	elon	google	is	musk	of	pichai	sundar	twitter

Preprocessing

❖ Vectorization

```
● ● ●  
  
# Tokenize samples  
def vectorize_text(sentence, tokenizer):  
    output = tokenizer.encode(sentence)  
    return torch.tensor(output.ids, dtype=torch.long)  
  
# Vectorize samples  
def vectorize_label(sentence_labels, sequence_length):  
    sentence_labels = sentence_labels + [5]*(sequence_length - len(sentence_labels))  
    return torch.tensor(sentence_labels, dtype=torch.long)  
  
# Vectorize the samples  
sentence_vecs = []  
label_vecs = []  
for sentence, sentence_labels in zip(corpus, labels):  
    sentence_vecs.append(vectorize_text(sentence, tokenizer))  
    label_vecs.append(vectorize_label(sentence_labels, sequence_length))
```

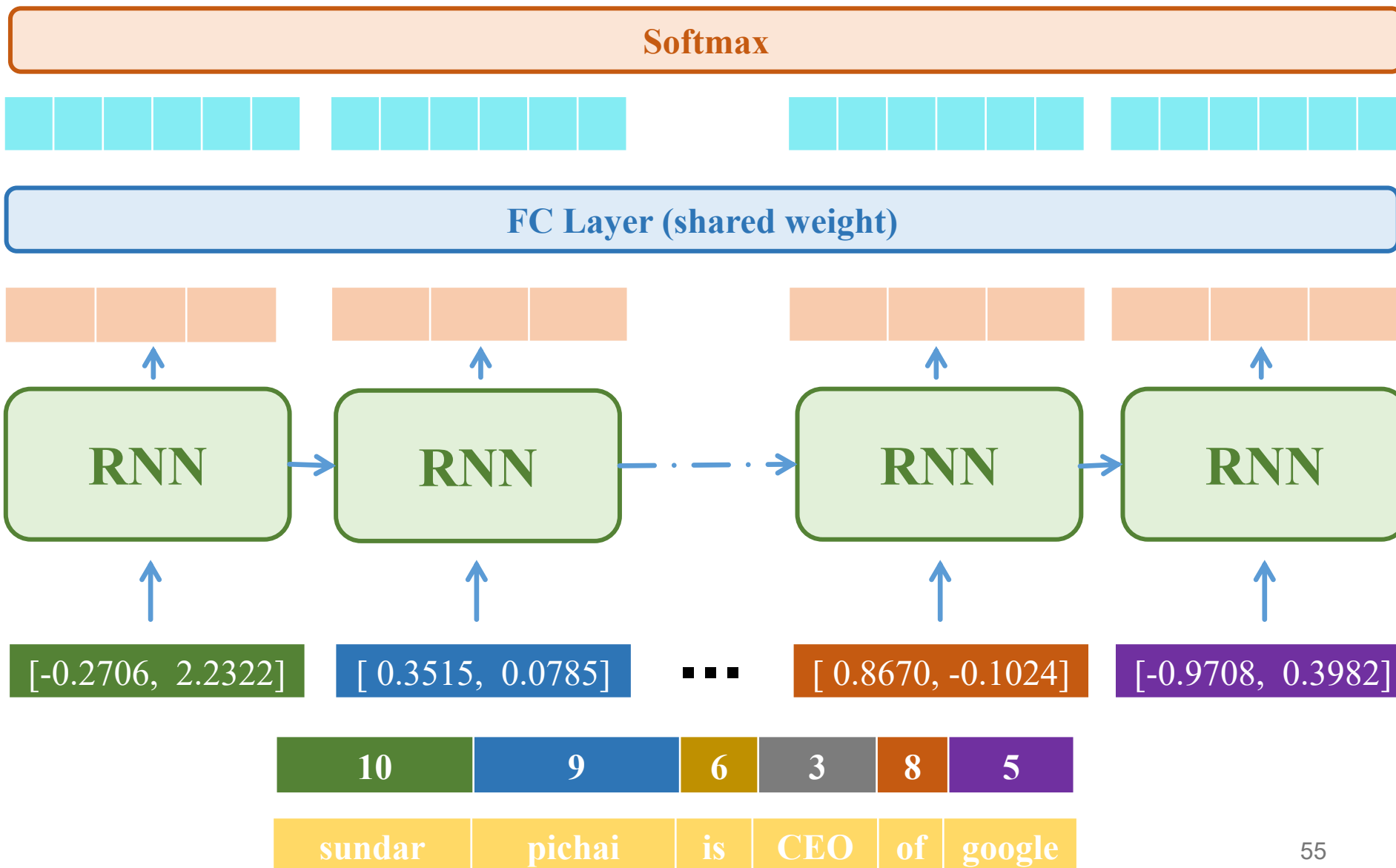
Embedding

```
nn.Embedding(vocab_size, 2)
```

0	<unk>	[-0.2367, 1.8109]
1	<pad>	[0.1966, -0.7150]
2	buy	[0.1041, 0.8893]
3	ceo	[-0.4212, -0.5279]
4	elon	[0.4374, -0.2224]
5	google	[-0.9708, 0.3982]
6	is	[0.1643, -0.5479]
7	musk	[0.1861, -1.8674]
8	of	[0.8670, -0.1024]
9	pichai	[0.3515, 0.0785]
10	sundar	[-0.2706, 2.2322]
11	twitter	[-1.1496, -0.4452]



NER



[-0.2367, 1.8109]

[0.1966, -0.7150]

[0.1041, 0.8893]

[-0.4212, -0.5279]

[0.4374, -0.2224]

[-0.9708, 0.3982]

[0.1643, -0.5479]

[0.1861, -1.8674]

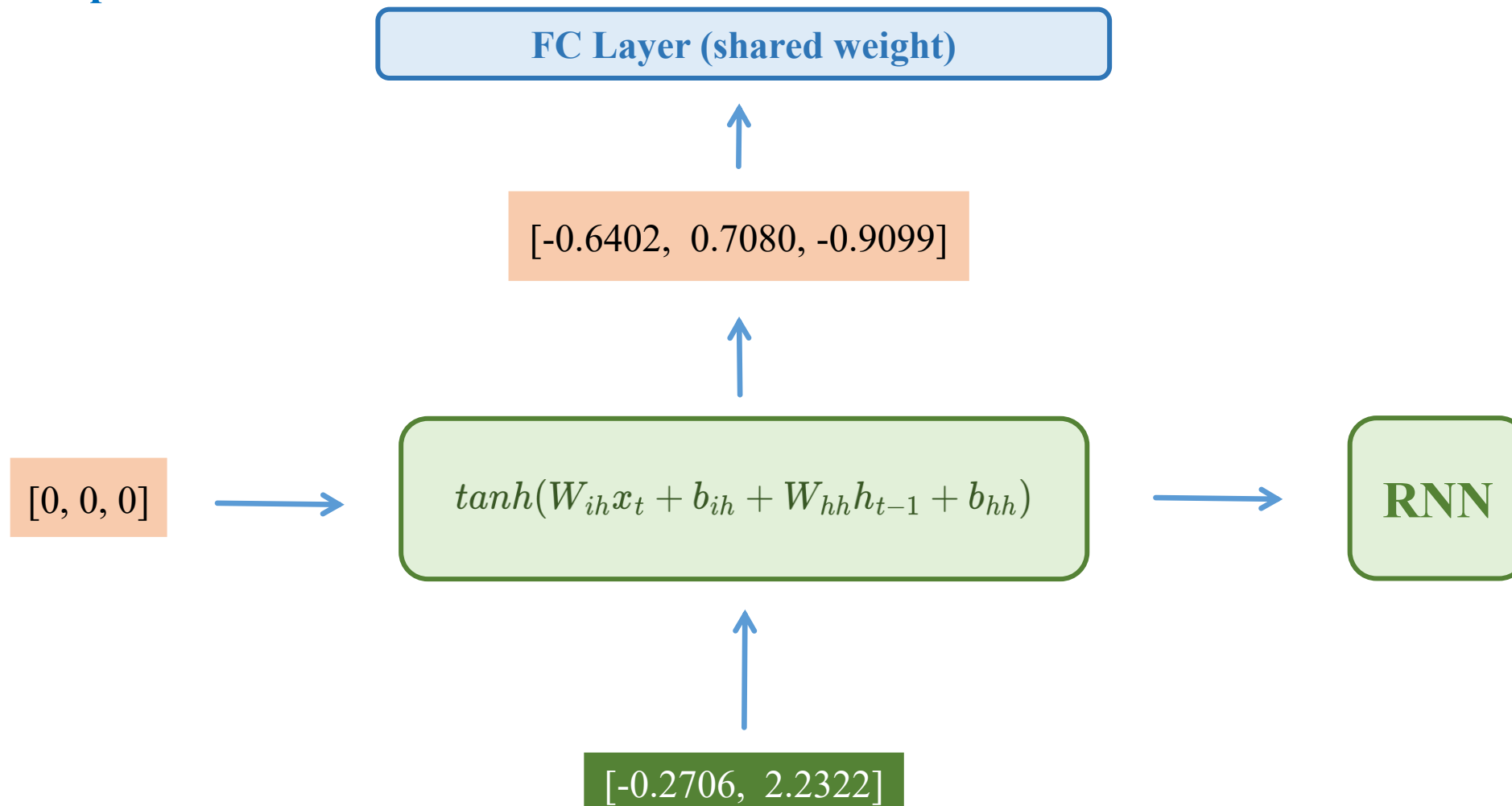
[0.8670, -0.1024]

[0.3515, 0.0785]

[-0.2706, 2.2322]

[-1.1496, -0.4452]

❖ 1 token computation



❖ 1 token to RNN

$$\begin{array}{c}
 \begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} [-0.4686, 0.5207, -0.1925], \\ [-0.3046, -0.4303, 0.3353], \\ [-0.2471, -0.3696, 0.3105] \end{bmatrix}^T = \begin{bmatrix} 0.0 & 0.0 & 0.0 \end{bmatrix} + \begin{bmatrix} -0.4083 & 0.0875 & -0.5172 \end{bmatrix}
 \end{array}$$

$\text{tanh}(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$

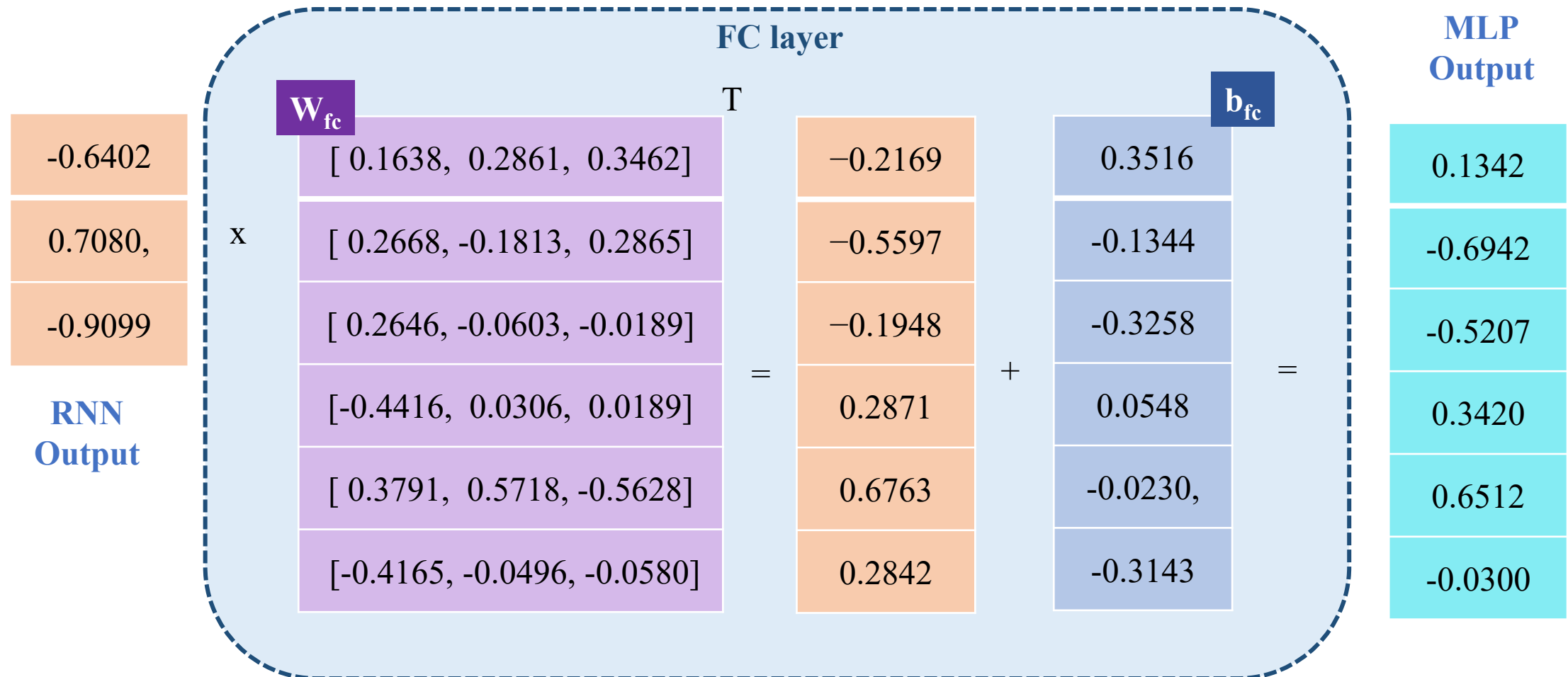
$\begin{bmatrix} -0.4083 & 0.0875 & -0.5172 \end{bmatrix}$

$$\begin{bmatrix} -0.6402 & 0.7080 & -0.9099 \end{bmatrix} = \tanh\left(\begin{bmatrix} -0.7583 & 0.8830 & -1.5267 \end{bmatrix}\right)$$

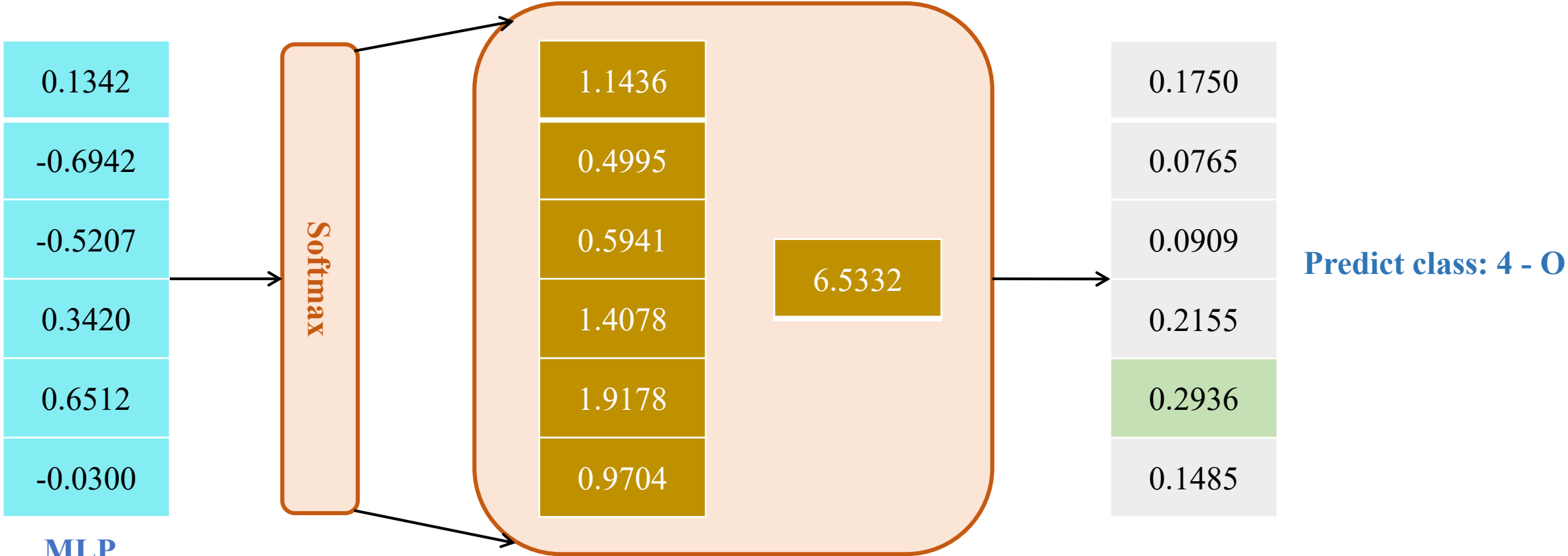
$$\begin{array}{c}
 \begin{bmatrix} -0.2706 & 2.2322 \end{bmatrix} \times \begin{bmatrix} [0.1107, -0.1420], \\ [0.5562, 0.4151], \\ [-0.0376, -0.4207] \end{bmatrix}^T = \begin{bmatrix} -0.3469 & 0.7760 & -0.9289 \end{bmatrix} + \begin{bmatrix} -0.0031 & 0.0195 & -0.0806 \end{bmatrix}
 \end{array}$$

$\begin{bmatrix} -0.3500 & 0.7956 & -1.0095 \end{bmatrix}$

❖ 1 token to MLP



❖ Softmax for MLP output



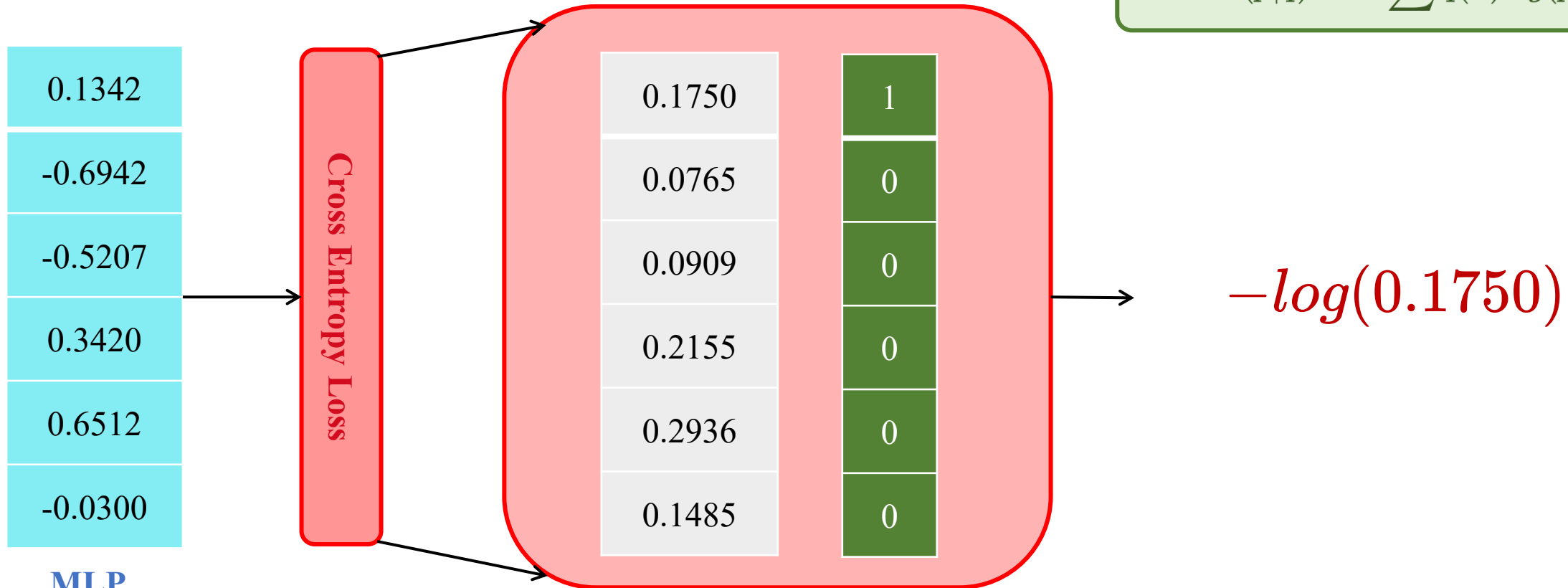
MLP
Output

0	1	2	3	4	5
B-Person	I-Person	B-Organization/Location	I-Organization/Location	O	<pad>

NER

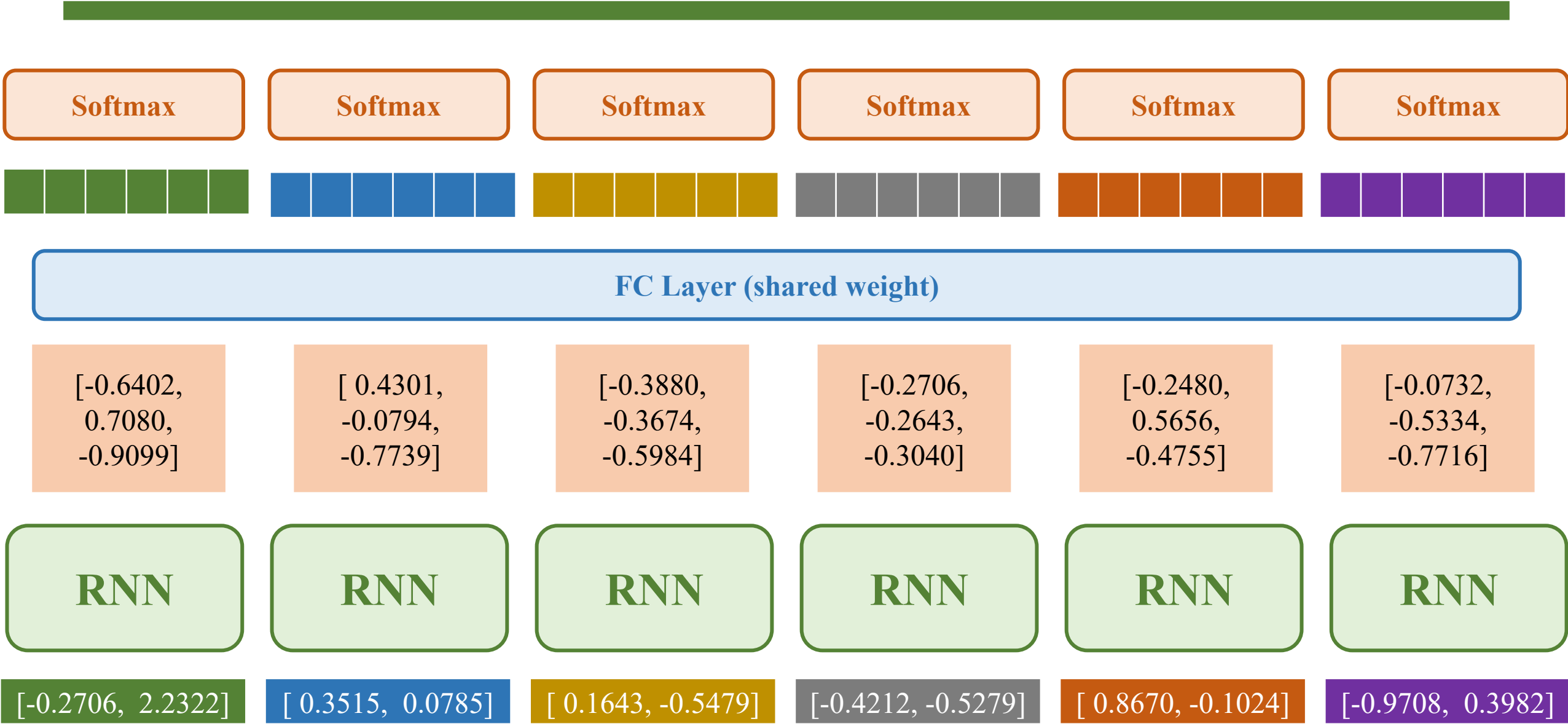
❖ nn.CrossEntropyLoss

$$H(p|q) = - \sum q(x) \log(p(x))$$



0	1	2	3	4	5
B-Person	I-Person	B-Organization/Location	I-Organization/Location	O	<pad>

NER



Question

